

Aula 00

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) Apresentação do Prof. Diego Carvalho - Informática	3
2) Metodologias de Desenvolvimento - Teoria	5
3) Metodologias de Desenvolvimento - Questões Comentadas - FGV	34
4) Metodologias de Desenvolvimento - Lista de Questões - FGV	39

APRESENTAÇÃO DO PROFESSOR

PROF. DIEGO CARVALHO

FORMADO EM CIÊNCIA DA COMPUTAÇÃO PELA
UNIVERSIDADE DE BRASÍLIA (UNB), PÓS-GRADUADO EM
GESTÃO DE TECNOLOGIA DA INFORMAÇÃO NA
ADMINISTRAÇÃO PÚBLICA E, ATUALMENTE, AUDITOR
FEDERAL DE FINANÇAS E CONTROLE DA SECRETARIA DO
TESOURO NACIONAL.

ESTRATÉGIA CONCURSOS

 PROFESSOR DIEGO CARVALHO - [WWW.INSTAGRAM.COM/PROFESSORDIEGOCARVALHO](https://www.instagram.com/professordiego-carvalho)



Sobre o curso: galera, todos os tópicos da aula possuem Faixas de Incidência, que indicam se o assunto cai muito ou pouco em prova. Diego, se cai pouco para que colocar em aula? Cair pouco não significa que não cairá justamente na sua prova! A ideia aqui é: se você está com pouco tempo e precisa ver somente aquilo que cai mais, você pode filtrar pelas incidências média, alta e altíssima; se você tem tempo sobrando e quer ver tudo, vejam também as incidências baixas e baixíssimas. *Fechado?*

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

INCIDÊNCIA EM PROVA: BAIXA

INCIDÊNCIA EM PROVA: MÉDIA

INCIDÊNCIA EM PROVA: ALTA

INCIDÊNCIA EM PROVA: ALTÍSSIMA

Além disso, essas faixas não são por banca – é baseado tanto na quantidade de vezes que caiu em prova independentemente da banca quanto nas minhas próprias avaliações sobre cada assunto.

#ATENÇÃO

Avisos Importantes



O curso abrange todos os níveis de conhecimento...

Esse curso foi desenvolvido para ser acessível a **alunos com diversos níveis de conhecimento diferentes**. Temos alunos mais avançados que têm conhecimento prévio ou têm facilidade com o assunto. Por outro lado, temos alunos iniciantes, que nunca tiveram contato com a matéria ou até mesmo que têm trauma dessa disciplina. A ideia aqui é tentar atingir ambos os públicos - iniciantes e avançados - da melhor maneira possível..



Por que estou enfatizando isso?



O **material completo** é composto de muitas histórias pessoais, exemplos, metáforas, piadas, memes, questões, desafios, esquemas, diagramas, imagens, entre outros. Já o **material simplificado** possui exatamente o mesmo núcleo do material completo, mas ele é menor e mais objetivo. *Professor, eu devo estudar por qual material?* Se você quiser se aprofundar nos assuntos ou tem dificuldade com a matéria, necessitando de um material mais passo-a-passo, utilize o material completo. Se você não quer se aprofundar nos assuntos ou tem facilidade com a matéria, necessitando de um material mais direto ao ponto, utilize o material simplificado.



Por fim...

O curso contém diversas questões espalhadas em meio à teoria. Essas questões possuem um comentário mais simplificado porque **têm o único objetivo de apresentar ao aluno como bancas de concurso cobram o assunto previamente administrado**. A imensa maioria das questões para que o aluno avalie seus conhecimentos sobre a matéria estão dispostas ao final da aula na lista de exercícios e **possuem comentários bem mais abrangentes**.

Sumário

Engenharia de Software	2
Conceitos Básicos	2
Processos de Desenvolvimento	6
Modelos Sequenciais	10
Modelo Iterativos	15
Modelo Especializados	28

ENGENHARIA DE SOFTWARE

Conceitos Básicos

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

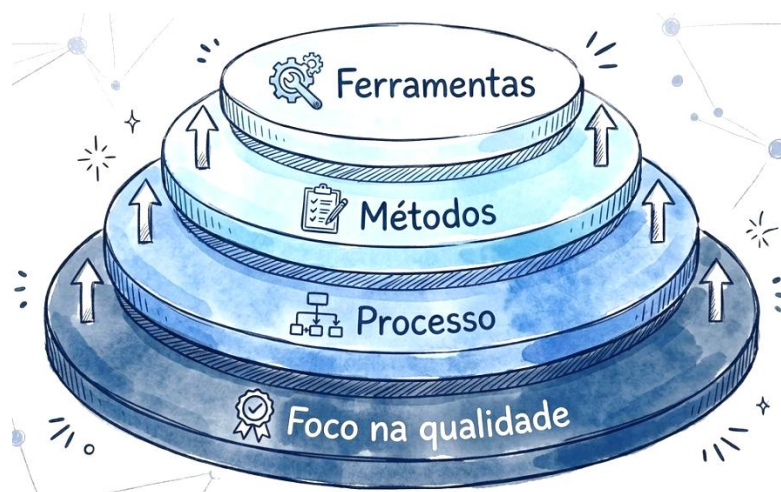
ENGENHARIA DE SOFTWARE

Disciplina da computação que aplica princípios, métodos e ferramentas de engenharia ao desenvolvimento, operação e manutenção de software, com o objetivo de produzir sistemas de qualidade, confiáveis e eficientes. Ela abrange atividades como levantamento de requisitos, projeto, implementação, testes e gestão, buscando garantir que o software atenda às necessidades dos usuários dentro de prazos e custos adequados.

Software não é apenas o programa executável, mas um conjunto mais amplo que inclui também documentação, dados de configuração e todos os elementos necessários para seu funcionamento adequado, conforme destacado por Ian Sommerville. Já a Engenharia de Software, segundo a IEEE, consiste na aplicação de uma abordagem sistemática, disciplinada e mensurável ao desenvolvimento, operação e manutenção de software, com o objetivo de garantir qualidade, organização e eficiência em todo o seu ciclo de vida.

A Engenharia de Software se distingue por ser sistemática, disciplinada e mensurável, indo muito além de apenas programar. Para Friedrich Bauer, ela aplica princípios de engenharia para produzir software econômico e confiável, enquanto Ian Sommerville destaca que envolve também gestão, métodos e ferramentas.

Já Roger Pressman a insere dentro da Engenharia de Sistemas, responsável por integrar hardware, software, processos e pessoas. Baseia-se em três pilares – processos, métodos e ferramentas – sustentados pelo foco na qualidade. Surgiu após a Crise do Software e evoluiu com abordagens estruturadas, orientação a objetos e, posteriormente, metodologias ágeis como Scrum e XP, que priorizam adaptação e colaboração.



Aqui cabe uma observação importante: Sommerville não enumera formalmente os princípios da Engenharia de Software, mas Pressman sim. Aqui vamos adotar a versão de Pressman e de Mandrioli:

PRINCÍPIOS (MANDRIOLI)

DESCRIÇÃO

FORMALIDADE	A formalidade exige que o desenvolvimento de software siga processos bem definidos, com registros claros das atividades realizadas. Isso reduz ambiguidades e evita falhas de comunicação entre equipes. Em vez de confiar apenas na comunicação informal, utiliza-se documentação estruturada para validar etapas. Esse rigor aumenta a confiabilidade do produto e diminui o risco de erros graves chegarem ao cliente final.
ABSTRAÇÃO	A abstração consiste em focar apenas nos aspectos essenciais de um problema, ignorando detalhes irrelevantes em um determinado momento. Essa abordagem permite simplificar a compreensão de sistemas complexos, facilitando a modelagem e o raciocínio. Ao criar diagramas ou modelos conceituais, por exemplo, o engenheiro destaca o que é importante sem se preocupar ainda com a implementação detalhada.
DECOMPOSIÇÃO	A decomposição envolve dividir um problema complexo em partes menores e mais simples de serem tratadas. Cada parte pode ser analisada, desenvolvida e testada de forma independente, o que facilita o controle e reduz a complexidade geral. Essa estratégia permite organizar melhor o desenvolvimento, distribuir tarefas entre equipes e aumentar a qualidade do software, tornando o sistema mais compreensível e gerenciável.
GENERALIZAÇÃO	A generalização busca criar soluções que não atendam apenas a um problema específico, mas que possam ser reutilizadas em diferentes contextos. Ao desenvolver componentes mais genéricos, reduz-se o retrabalho e aumenta-se a eficiência no desenvolvimento. Esse princípio está diretamente ligado à reutilização de software, permitindo a criação de bibliotecas e módulos que podem ser aplicados em diversos projetos distintos.
FLEXIBILIZAÇÃO	A flexibilização refere-se à capacidade do software de ser modificado com facilidade, sem comprometer seu funcionamento. Como sistemas estão sujeitos a mudanças constantes, seja por correção de erros ou novas demandas, é essencial que sejam projetados para evoluir. Esse princípio está relacionado à manutenibilidade, permitindo adaptações rápidas e seguras, prolongando a vida útil e a relevância do software ao longo do tempo.

CATEGORIA (PRESSMAN)	PRINCÍPIO	DESCRIÇÃO
PRINCÍPIOS FUNDAMENTAIS (ORIENTAM A PRÁTICA COMO UM TODO)	A razão de existir	Todo sistema de software deve agregar valor aos seus usuários; todas as decisões devem considerar se estão trazendo um benefício real.
	KISS (<i>Keep It Simple, Stupid!</i>)	Todo projeto deve ser o mais simples possível (sem ser simplista), o que resulta em um software menos propenso a erros e mais fácil de compreender e manter.
	Mantenha a visão	É essencial manter uma visão arquitetural clara e consistente para que o projeto não se torne uma "colcha de retalhos" incompatível.
	O que um produz outros consomem	Deve-se especificar, projetar e codificar lembrando sempre que outra pessoa (ou você mesmo no futuro) precisará entender, testar ou dar manutenção ao artefato gerado.
	Esteja aberto para o futuro	Sistemas de sucesso são projetados para se adaptarem a mudanças e evoluírem, resolvendo o problema geral e estendendo sua vida útil.
	Planeje visando a reutilização	Criar componentes genéricos e planejá-los com antecedência reduz o esforço futuro, economiza tempo e aumenta o valor do sistema.
	Pense!	Refletir e analisar de forma clara antes de agir (projetar ou codificar) produz resultados superiores e reduz retrabalho.

PRINCÍPIOS DO PROCESSO (ORIENTAM A ORGANIZAÇÃO DO TRABALHO)	Seja ágil	Mantenha a abordagem técnica e os produtos o mais simples possível, com foco na economia de ações e tomadas de decisão locais.
	Concentre-se na qualidade	A qualidade do produto deve ser o foco principal em toda e qualquer atividade, ação e tarefa do processo de software.
	Esteja pronto para adaptações	Não há espaço para dogmas; o processo deve ser adaptado às restrições do problema, da equipe e do projeto.
	Monte uma equipe eficiente	Pessoas são o fator mais importante; crie uma equipe auto-organizada baseada em confiança e respeito mútuos.
	Estabeleça comunicação	É fundamental coordenar esforços para evitar a omissão de informações críticas entre os envolvidos.
	Gerencie Mudanças	Crie mecanismos formais ou informais para avaliar, aprovar e acomodar as mudanças solicitadas ao longo do processo.
	Avalie os riscos	Muitas coisas podem dar errado; identifique os riscos antecipadamente e crie planos de contingência.
	Gere artefatos de valor	Só produza artefatos (modelos, listas, etc) se eles agregarem valor e transmitirem informações claras para as próximas etapas.
PRINCÍPIOS DA PRÁTICA (ORIENTAM O TRABALHO TÉCNICO)	Divida e conquiste	Utilize a "separação por interesses" para dividir problemas complexos em partes menores e independentes, facilitando a resolução.
	Compreenda a abstração	Use a abstração para focar na essência do problema, ignorando detalhes de baixo nível e facilitando a comunicação do significado.
	Esforce-se pela consistência	Estabeleça um contexto previsível e padronizado (como esquemas de cores e navegação em interfaces) para facilitar o uso do software.
	Foco na transferência de informações	Preste atenção redobrada ao projetar e testar interfaces (internas e externas), pois é onde o fluxo de informações costuma gerar erros ou omissões.
	Construa modularidade efetiva	Crie módulos ou componentes que sejam altamente coesos (tenham uma única função bem definida) e apresentem baixo acoplamento.
	Verifique os padrões	Empregue padrões de projeto comprovados para resolver problemas recorrentes, aproveitando o conhecimento já consolidado.
	Múltiplas perspectivas	Represente problemas e soluções sob diferentes pontos de vista (cenários, classes, comportamentos) para revelar erros que ficariam ocultos.

Facilite a manutenção

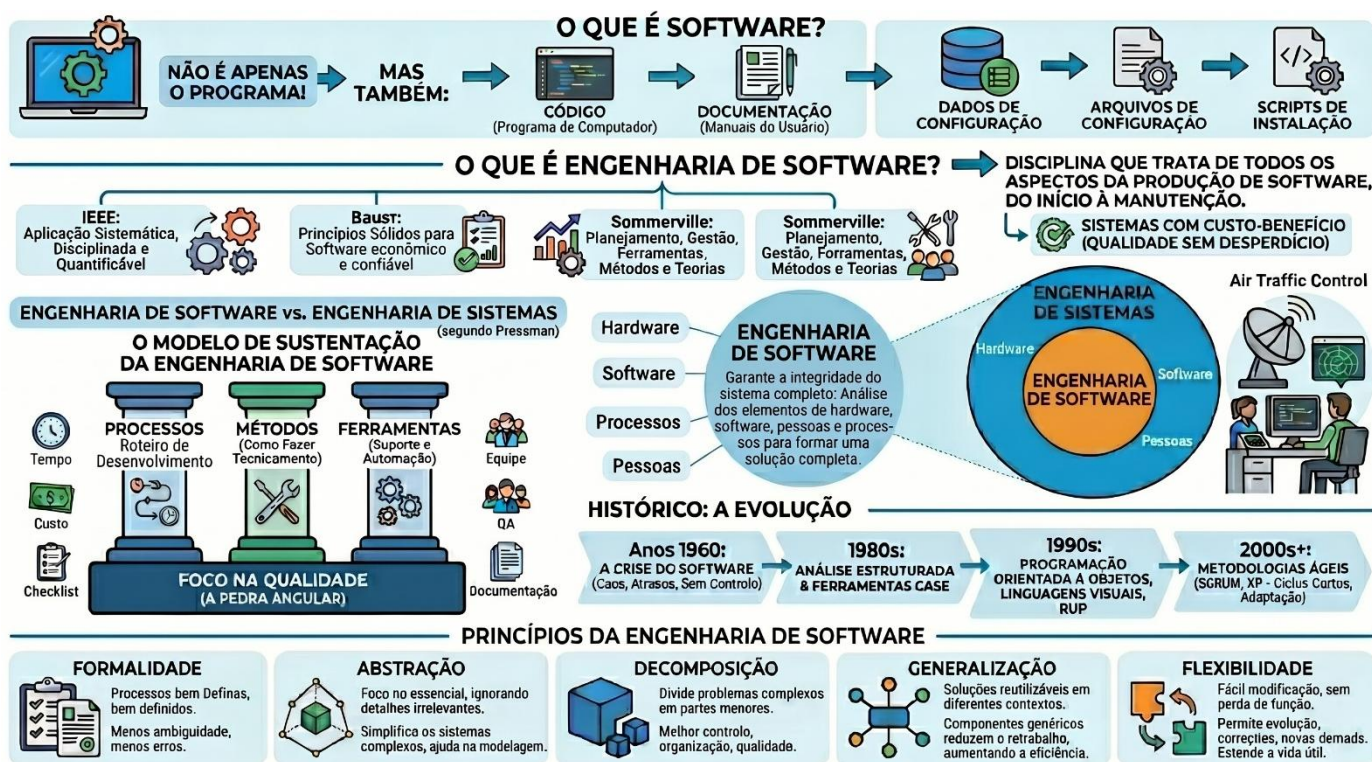
Projete o código assumindo que erros serão descobertos e novas funções serão pedidas a longo prazo, aplicando práticas que ajudem as futuras equipes de manutenção.

Outro ponto que merece destaque é que a Engenharia de Software não termina quando o sistema entra em produção. "Em produção" significa que o software já está disponível no ambiente real do cliente – mas isso não é o fim da história. Sommerville é bastante enfático ao tratar o desenvolvimento de software como um processo contínuo e iterativo: sistemas evoluem, requisitos mudam, erros são descobertos, e o ambiente ao redor do software se transforma. Por isso, a manutenção e a evolução do sistema são atividades tão importantes quanto o desenvolvimento inicial.

Vamos resumir tudo? Software = Programa + Documentação + Dados de Configuração. Engenharia de Software é a disciplina que cuida de todos os aspectos da produção de software, com foco em qualidade e boa relação custo-benefício. Ela está inserida no contexto maior da Engenharia de Sistemas, que abrange hardware, software e processos; e ela se apoia em processos, métodos e ferramentas, tendo a qualidade como pedra fundamental.

Ela surgiu na década de 1960 para contornar a Crise do Software, e evoluiu até as metodologias ágeis (a partir de 2001). Seus princípios fundamentais são: Formalidade, Abstração, Decomposição, Generalização e Flexibilização – oriundos principalmente de Ghezzi et al., mas amplamente adotados na literatura da área. O ciclo de vida do software é contínuo – não termina com a entrega. Ficou claro? Esses conceitos são a base de tudo que vem a seguir em Engenharia de Software. Vamos continuar...

A Engenharia de Software não se encerra com a entrada do sistema em produção, pois o software é continuamente ajustado, evoluído e mantido diante de mudanças e novos requisitos, como destaca Ian Sommerville. Em síntese, software engloba programa, documentação e dados de configuração, enquanto a Engenharia de Software abrange todos os aspectos de sua produção, com foco em qualidade e custo-benefício, inserida no contexto mais amplo da Engenharia de Sistemas.



Sustenta-se em processos, métodos e ferramentas, tendo a qualidade como base, surgiu para enfrentar a Crise do Software e evoluiu até as metodologias ágeis. Seus princípios incluem formalidade, abstração, decomposição, generalização e flexibilização, reforçando que o ciclo de vida do software é contínuo e não termina com a entrega.

Processos de Desenvolvimento

PROCESSOS DE DESENVOLVIMENTO DE SOFTWARE

Conjuntos organizados de atividades, métodos, práticas e ferramentas utilizados para planejar, construir, testar, entregar e manter sistemas de software. Eles definem como as etapas do desenvolvimento são estruturadas, a ordem em que ocorrem, os papéis envolvidos e os artefatos produzidos, com o objetivo de garantir qualidade, previsibilidade e alinhamento com as necessidades do cliente, podendo variar de modelos mais rígidos a abordagens ágeis e adaptativas.

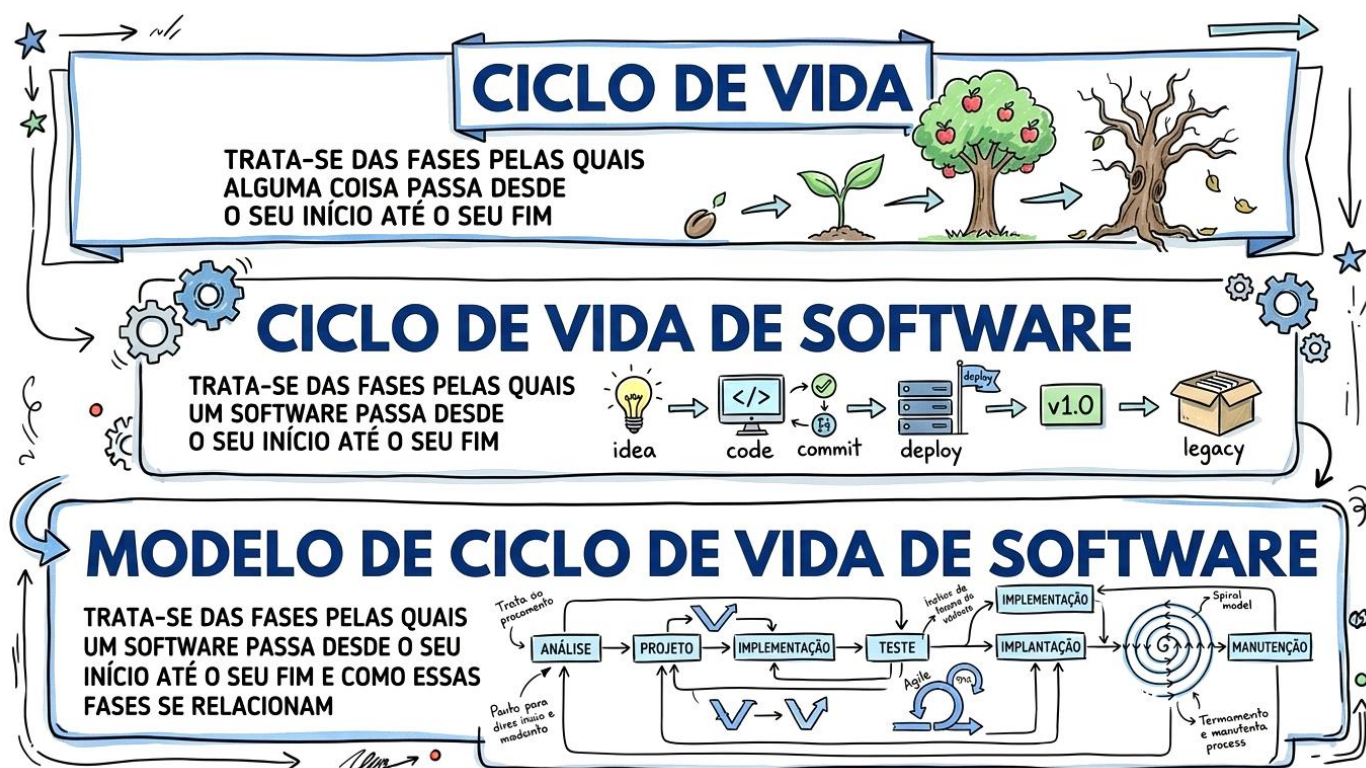
O termo “ciclo de vida” vem da biologia e descreve as fases pelas quais algo passa do início ao fim, ideia que foi adaptada para áreas como organizações, projetos e produtos. Na Engenharia de Software, representa o conjunto de etapas desde a concepção até a retirada do sistema, sendo entendido como uma sequência de atividades relacionadas à criação e evolução do software, como destaca Steve McConnell.

Apesar de o conceito ser simples, não há consenso entre autores sobre quais são exatamente essas fases – inclusive Roger Pressman apresenta variações entre edições de sua obra. Na prática, o mais importante é compreender a lógica e a ordem das principais etapas, sem tentar memorizar todos os modelos existentes. Dito isso, vamos ver oito fases de um ciclo de vida bem completo e vamos entender o que acontece em cada etapa do desenvolvimento:

ETAPAS	DESCRIÇÃO
PLANEJAMENTO	O planejamento de projeto estabelece a base para estimar recursos, custos e prazos de forma realista. A partir de um escopo inicial e de um esboço de requisitos, elabora-se um plano que define o processo de desenvolvimento. Esse plano deve ser continuamente refinado ao longo do projeto, especialmente ao final de cada fase. Inclui atividades de acompanhamento e controle, essenciais para a gerência eficaz do projeto.
ANÁLISE E ESPECIFICAÇÃO DE REQUISITOS	Nesta fase, ocorre o levantamento e refinamento dos requisitos do sistema. O engenheiro de software busca compreender o domínio do problema, restrições, objetivos e funcionalidades esperadas, utilizando técnicas como entrevistas. Os requisitos coletados são modelados, analisados e documentados. O foco está em definir o que o sistema deve fazer, sem detalhar ainda como será implementado.
PROJETO	Essa fase transforma requisitos em uma solução técnica estruturada. Inicialmente, define-se a arquitetura do sistema, identificando seus principais componentes e organização geral. Em seguida, realiza-se o projeto detalhado, refinando cada componente até um nível que permita sua implementação. Essa etapa considera tecnologias, padrões e decisões estruturais necessárias para viabilizar o desenvolvimento do software.
IMPLEMENTAÇÃO	Nesta fase, o projeto detalhado é convertido em código executável. Cada componente definido anteriormente é desenvolvido utilizando linguagens e ferramentas adequadas. O objetivo é transformar os modelos e especificações em um sistema funcional. Essa etapa exige aderência ao projeto, boas práticas de programação e organização do código, garantindo que o software possa ser posteriormente testado e integrado.
TESTES	A fase de testes visa verificar e validar o software desenvolvido. Inicialmente, são realizados testes de unidade em cada componente individual. Em seguida, ocorre a integração progressiva dos módulos, com testes de integração. Por fim, o sistema

	completo é submetido a testes de sistema. O objetivo é identificar falhas, garantir qualidade e verificar se o software atende aos requisitos definidos.
ENTREGA E IMPLANTAÇÃO	Após os testes, o software é disponibilizado no ambiente de produção. Essa fase envolve atividades como instalação, configuração do ambiente, treinamento dos usuários e, quando necessário, migração de dados. Também são realizados testes de aceitação para verificar se o sistema atende às expectativas do cliente. Uma vez validado, o software é oficialmente aceito e passa a operar em condições reais.
OPERAÇÃO	Na fase de operação, o software é utilizado em ambiente real pelos usuários finais. O sistema passa a executar suas funções no contexto para o qual foi desenvolvido, apoiando atividades organizacionais. Nesse momento, são observados aspectos como desempenho, usabilidade e estabilidade. Essa fase fornece insumos importantes para futuras melhorias e ajustes no sistema.
MANUTENÇÃO	A manutenção ocorre após a entrega e envolve modificações no software. Essas mudanças podem corrigir erros, adaptar o sistema a novos ambientes ou adicionar funcionalidades. Dependendo da complexidade, pode ser necessário reiniciar parte do processo de desenvolvimento. A manutenção é contínua e essencial para garantir a evolução e a adequação do software às necessidades dos usuários.

Alguns autores ampliam o ciclo de vida incluindo fases como análise (modelagem do problema), homologação (aceite formal do cliente) e gerência de configuração (controle de versões e artefatos). Um ponto crucial é diferenciar ciclo de vida de modelo de ciclo de vida: o primeiro descreve apenas as fases pelas quais o software passa do início ao fim, enquanto o segundo define, além das fases, como elas se relacionam. Por exemplo, no modelo em cascata, as etapas ocorrem de forma sequencial e rígida – uma só começa após o término da anterior –, enquanto no desenvolvimento iterativo essas mesmas atividades se repetem em ciclos, com maior flexibilidade e adaptação.



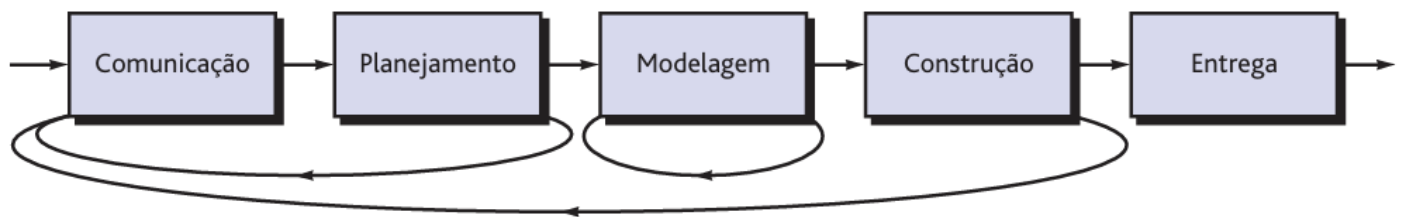
Em provas, é comum tratar modelo de ciclo de vida e modelo de processo como sinônimos. Um processo de software é o conjunto de atividades, ações e tarefas necessárias para criar um produto de software, organizadas dentro de um modelo ou metodologia que define como elas se relacionam. Segundo Roger Pressman, uma metodologia genérica envolve cinco atividades principais – comunicação, planejamento,

modelagem, construção e entrega – além de atividades de apoio, como controle do projeto, gestão de riscos, garantia da qualidade e gerenciamento de configuração. Ele também destaca o fluxo do processo, que determina a ordem e a dinâmica dessas atividades ao longo do tempo.

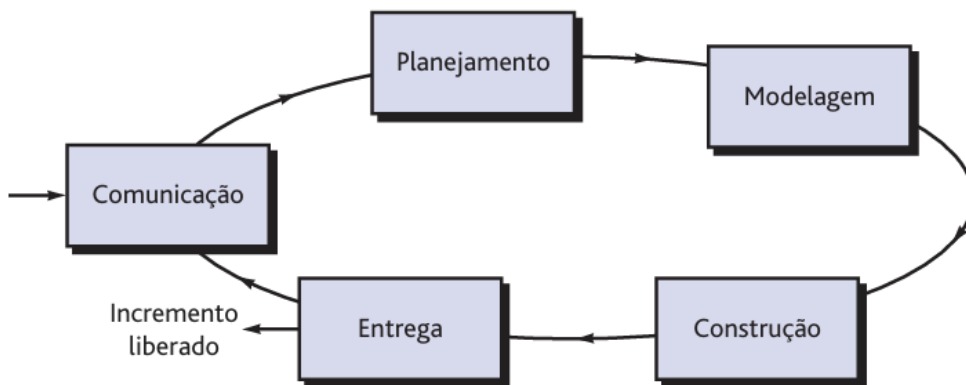
Um fluxo de processo linear executa cada uma das cinco atividades metodológicas em sequência, começando com a comunicação e culminando com a entrega.



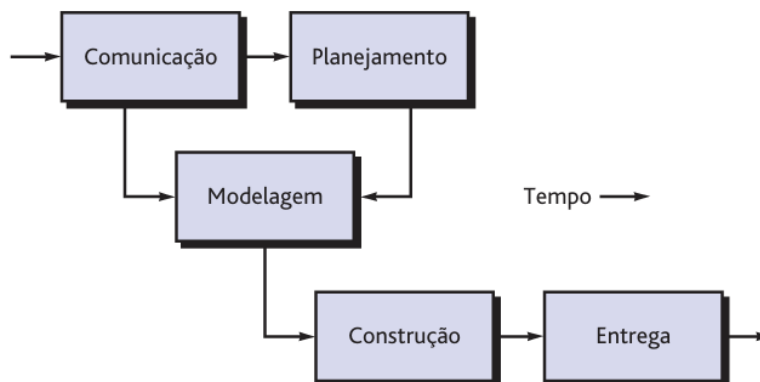
Já um fluxo de processo iterativo repete uma ou mais das atividades antes de prosseguir para a atividade seguinte conforme a imagem seguinte:



Continuando, um fluxo de processo evolucionário executa as atividades de forma "circular", sendo que cada volta pelas cinco atividades conduz a uma versão mais completa do software.

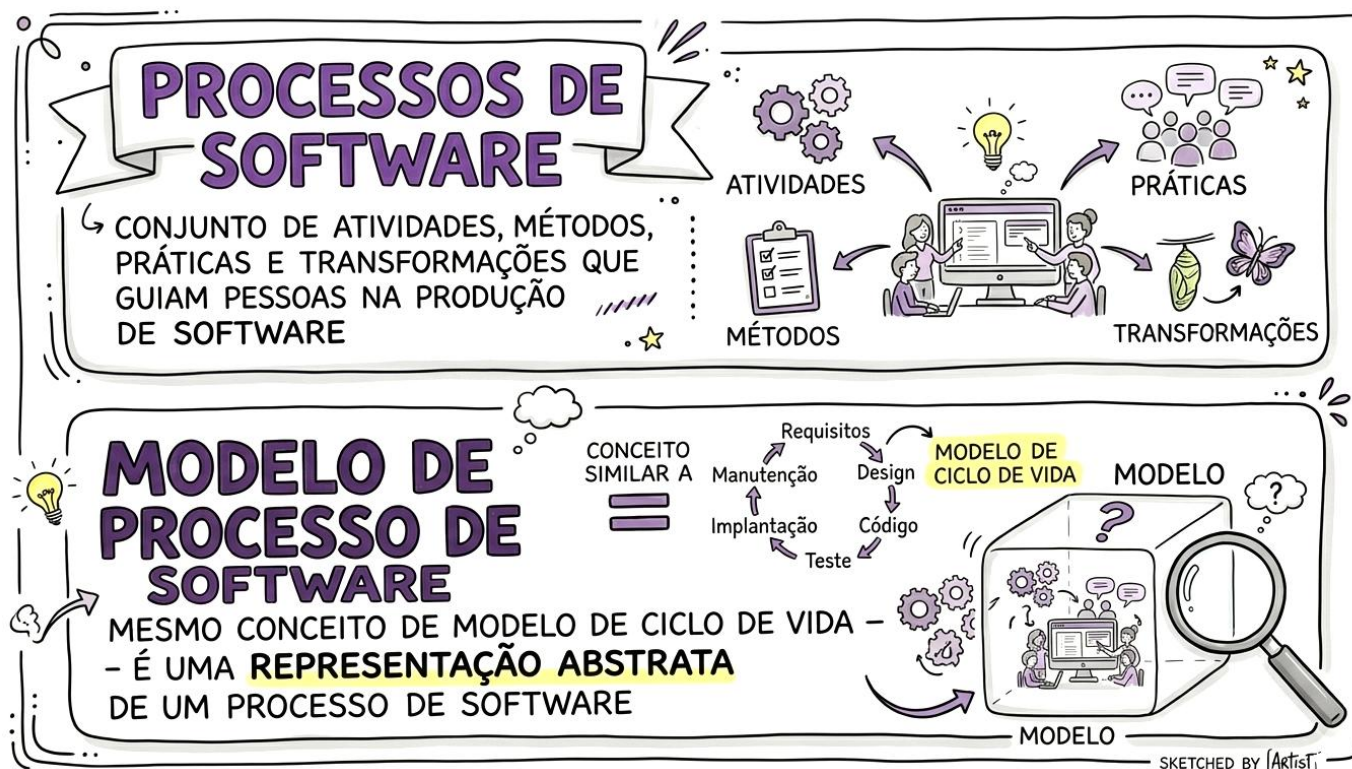


Por fim, um fluxo de processo paralelo executa uma ou mais atividades em paralelo com outras (Ex: a modelagem para um aspecto do software poderia ser executada em paralelo com a construção de outro):



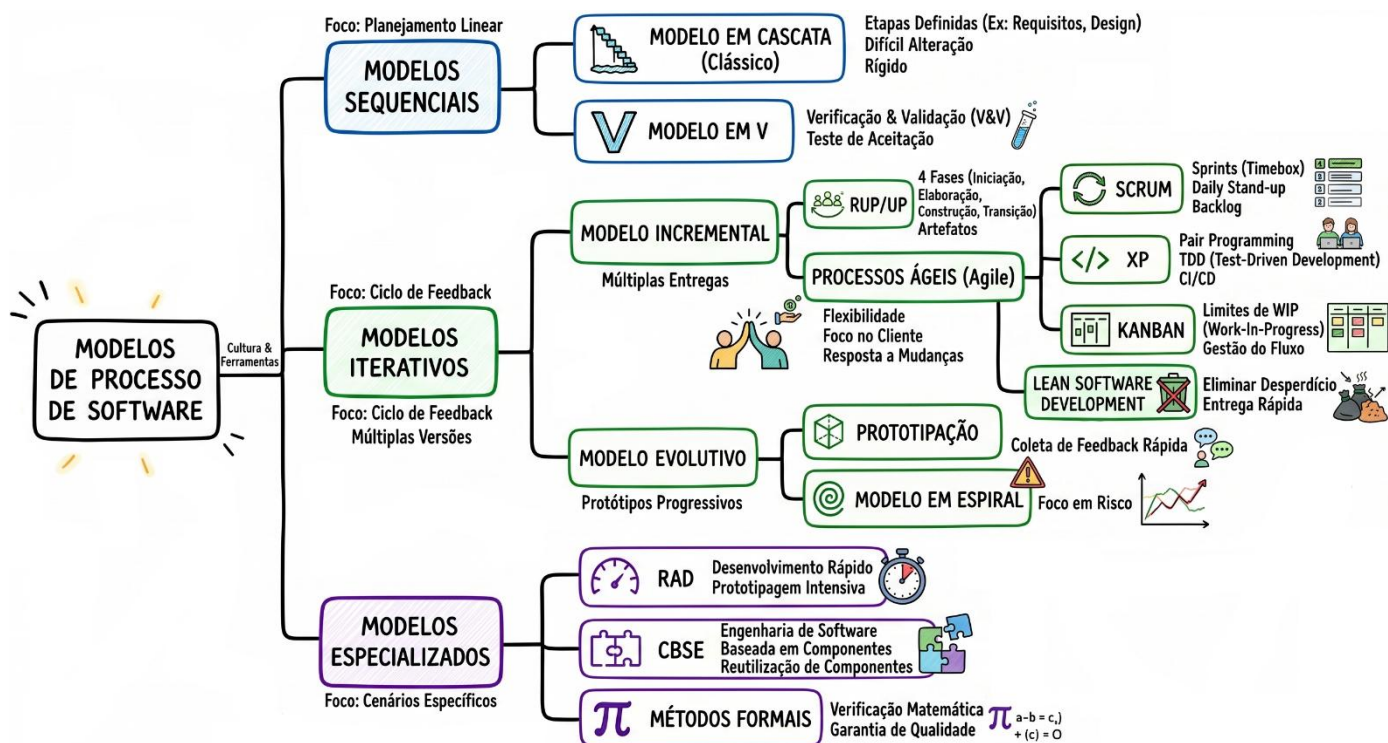
Note que cada etapa pode incluir diversas tarefas e gerar documentos específicos, isto é, quanto mais complexo o projeto, mais estruturado e detalhado precisa ser o processo de comunicação para garantir que todos estejam alinhados e que o software atenda às expectativas. O processo completo inclui: a escolha de um modelo de ciclo de vida; o detalhamento de suas macro-atividades; a escolha de métodos, técnicas e procedimentos; e a definição de recursos e artefatos necessários.

Já segundo Sommerville, um processo de software é um conjunto de atividades e resultados associados que produz um produto de software. Para ele, existem quatro atividades fundamentais presentes em todos os processos de software: Especificação de Software (que define o que o sistema deve fazer); Desenvolvimento de Software (que projeta e implementa o sistema); Validação de Software (que garante que o sistema atende às necessidades do cliente); Evolução de Software (que modifica o sistema para acompanhar mudanças).



Dizer que modelo de ciclo de vida e processo de software são iguais é uma simplificação, pois o processo é mais amplo e engloba o modelo como um de seus componentes, além de considerar contexto, tecnologia e equipe. Segundo Ian Sommerville, os processos podem ser agrupados em cascata, iterativo e baseado em componentes, enquanto outras classificações os organizam como sequenciais, incrementais e evolutivos – visões diferentes, mas complementares.

Já a metodologia (ou processo de desenvolvimento) define de forma mais concreta o que fazer, como e quando fazer, sendo uma instância prática do processo. Em síntese, o processo é mais abstrato e abrangente, enquanto a metodologia é sua aplicação estruturada no desenvolvimento. Ficou claro? Esses conceitos são a base para entender os modelos de processo que vamos estudar a seguir. Tudo vai fazer sentido a partir daqui. Vamos em frente...



Modelos Sequenciais

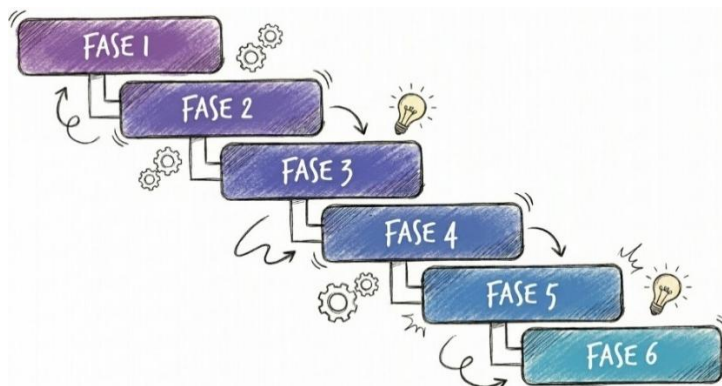
Modelo em Cascata

INCIDÊNCIA EM PROVA: ALTÍSSIMA

MODELO EM CASCATA

Modelo de desenvolvimento de software sequencial e linear, no qual as fases – como levantamento de requisitos, análise, projeto, implementação, testes e manutenção – são executadas uma após a outra, sem retorno significativo às etapas anteriores. Cada fase deve ser concluída antes do início da próxima, com forte ênfase em planejamento e documentação. É mais adequado para projetos com requisitos bem definidos e estáveis, mas apresenta baixa flexibilidade diante de mudanças.

O modelo em cascata – também chamado de clássico, sequencial, linear, tradicional, waterfall, rígido, top-down ou monolítico – recebe esse nome pela execução encadeada das fases, que fluem de forma linear, sem retorno ou sobreposição. Ele é atribuído a Winston Royce, que, curiosamente, não o propôs como ideal, mas sim destacou suas limitações e riscos, especialmente em projetos complexos. Sua regra central é a rigidez: cada fase só começa após a conclusão e aprovação total da anterior, o que torna o processo previsível, porém pouco flexível a mudanças.

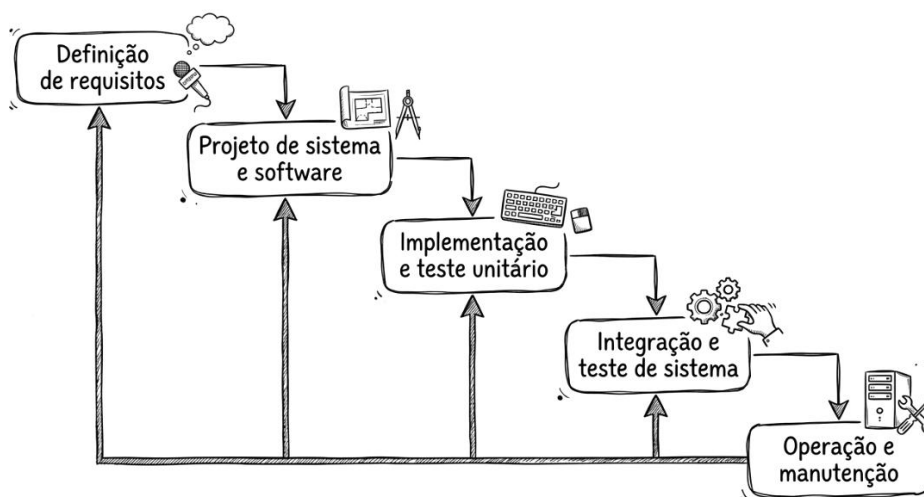


Assim como vimos no ciclo de vida geral, aqui também não há consenso entre os autores sobre quais são exatamente as fases. Cada um define as suas. Veja a comparação:

POR SOMMERVILLE	POR ROYCE	POR PRESSMAN (4ª ED.)	POR PRESSMAN (6ª ED.)
Análise e Definição de Requisitos	Requisitos de Sistema	Modelagem e Engenharia do Sistema	Comunicação
Projeto de Sistema e Software	Requisitos de Software	Análise de Requisitos de Software	Planejamento
Implementação e Teste de Unidade	Análise	Projeto	Modelagem
Integração e Teste de Sistema	Projeto	Geração de Código	Construção
Operação e Manutenção	Codificação	Teste e Manutenção	Implantação
	Teste		
	Operação		

Percebam as grandes diferenças! E reparem que o próprio Pressman mudou de ideia entre a 4ª e a 6ª edição – isso já gerou polêmica em provas. Infelizmente, é assim: quando uma questão cobrar as fases sem dizer qual autor ou edição, podemos ter ambiguidade. Sempre leiam o enunciado com atenção para identificar a referência.

De acordo com Ian Sommerville, o Modelo em Cascata é um exemplo de processo dirigido a planos (plan-driven process). O que isso significa? Significa que, em princípio, você deve planejar e programar todas as atividades antes de começar a trabalhar nelas. Tudo é definido com antecedência – escopo, requisitos, arquitetura – antes de avançar para a próxima fase. As fases, segundo Sommerville, são:

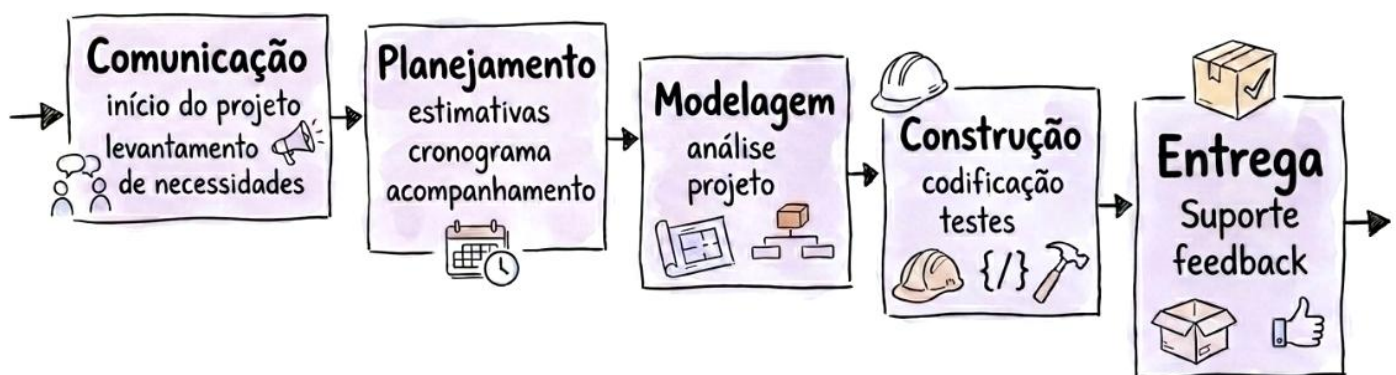


ETAPAS	DESCRIÇÃO
ANÁLISE E DEFINIÇÃO DE REQUISITOS	Os serviços, restrições e metas do sistema são estabelecidos por meio de consulta aos usuários e documentados como especificação do sistema.
PROJETO DE SISTEMA E SOFTWARE	Os requisitos são alocados para componentes de hardware e software. Define-se a arquitetura geral do sistema e as abstrações fundamentais do software.

IMPLEMENTAÇÃO E TESTE DE UNIDADE	O projeto é desenvolvido como um conjunto de programas ou unidades. Cada unidade é verificada para garantir que atende à sua especificação.
INTEGRAÇÃO E TESTE DE SISTEMA	As unidades são integradas e testadas como sistema completo. Após os testes, o software é entregue ao cliente.
OPERAÇÃO E MANUTENÇÃO	O sistema é instalado e colocado em uso. A manutenção envolve correção de erros, melhorias de implementação e ampliação de serviços conforme novos requisitos surgem. Normalmente é a fase mais longa do ciclo de vida.

Segundo Ian Sommerville, no modelo em cascata cada fase é formalmente aprovada antes da próxima, mas, na prática, há sobreposição e feedback entre etapas, o que gera retrabalho não planejado, já que o modelo não estrutura essas interações. Para evitar custos, equipes acabam congelando requisitos cedo demais, o que pode levar a sistemas que não atendem às reais necessidades.

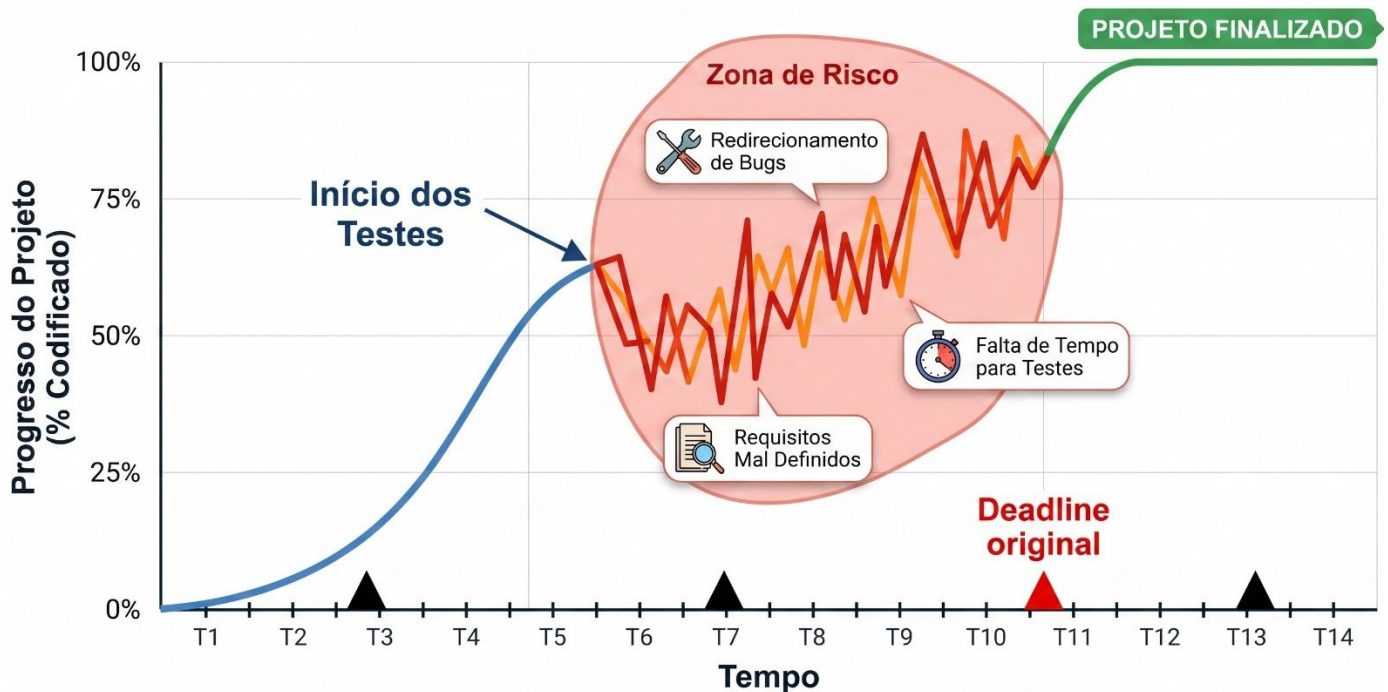
O próprio Winston Royce já previa feedback entre fases adjacentes, sendo a rigidez extrema resultado de adoções posteriores. Já Roger Pressman destaca que o cascata ainda é útil quando os requisitos são claros e estáveis, como em adaptações bem definidas ou projetos com baixo grau de mudança, descrevendo-o como um processo sequencial que vai de requisitos até suporte contínuo. De acordo com Pressman, essas são as fases do Modelo em Cascata:



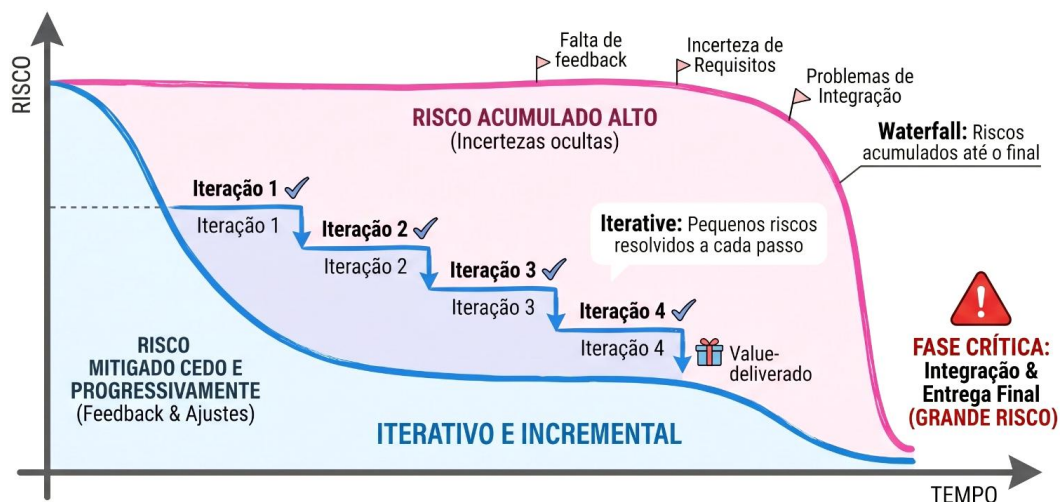
O principal problema do modelo em cascata é atrasar a identificação e a redução de riscos, já que erros cometidos nas fases iniciais – como requisitos mal definidos ou decisões arquiteturais equivocadas – só tendem a ser descobertos nas etapas finais, especialmente nos testes. Como as fases são rígidas e sequenciais, a correção desses problemas ocorre tardiamente, tornando-se muito mais cara, complexa e, às vezes, inviável. É por isso que esse modelo é frequentemente associado a alto retrabalho e baixa capacidade de adaptação a mudanças.

Pressman cita estudos empíricos que mostram uma realidade assustadora: o custo de corrigir um erro identificado na fase de manutenção pode ser de 50 a 200 vezes maior do que se o mesmo erro tivesse sido identificado ainda na fase de requisitos. Repitam isso para si mesmos: cinquenta a duzentas vezes mais caro. Isso por si só justifica a busca por modelos que reduzam riscos desde o início.

ANÁLISE DE PROGRESSO DE PROJETO: GESTÃO DE RISCOS E TESTES



O custo de correção de erros depende muito mais de quando o problema é descoberto do que de quando ele surgiu: falhas identificadas cedo são baratas, mas, se detectadas apenas no final, exigem grande retrabalho. Modelos iterativos reduzem riscos desde o início ao permitir feedback contínuo, enquanto no cascata os riscos se acumulam até as fases finais. Além disso, segundo Roger Pressman, o modelo apresenta limitações como dificuldade em lidar com mudanças, exigência irreal de requisitos completos no início, ausência de entregas parciais ao cliente, criação de estados de bloqueio entre equipes e inadequação ao ritmo dinâmico do desenvolvimento moderno.



Apesar de todas as críticas, o Modelo em Cascata não deve ser descartado completamente. Tanto Sommerville quanto Pressman concordam que ele ainda tem aplicação em contextos específicos:

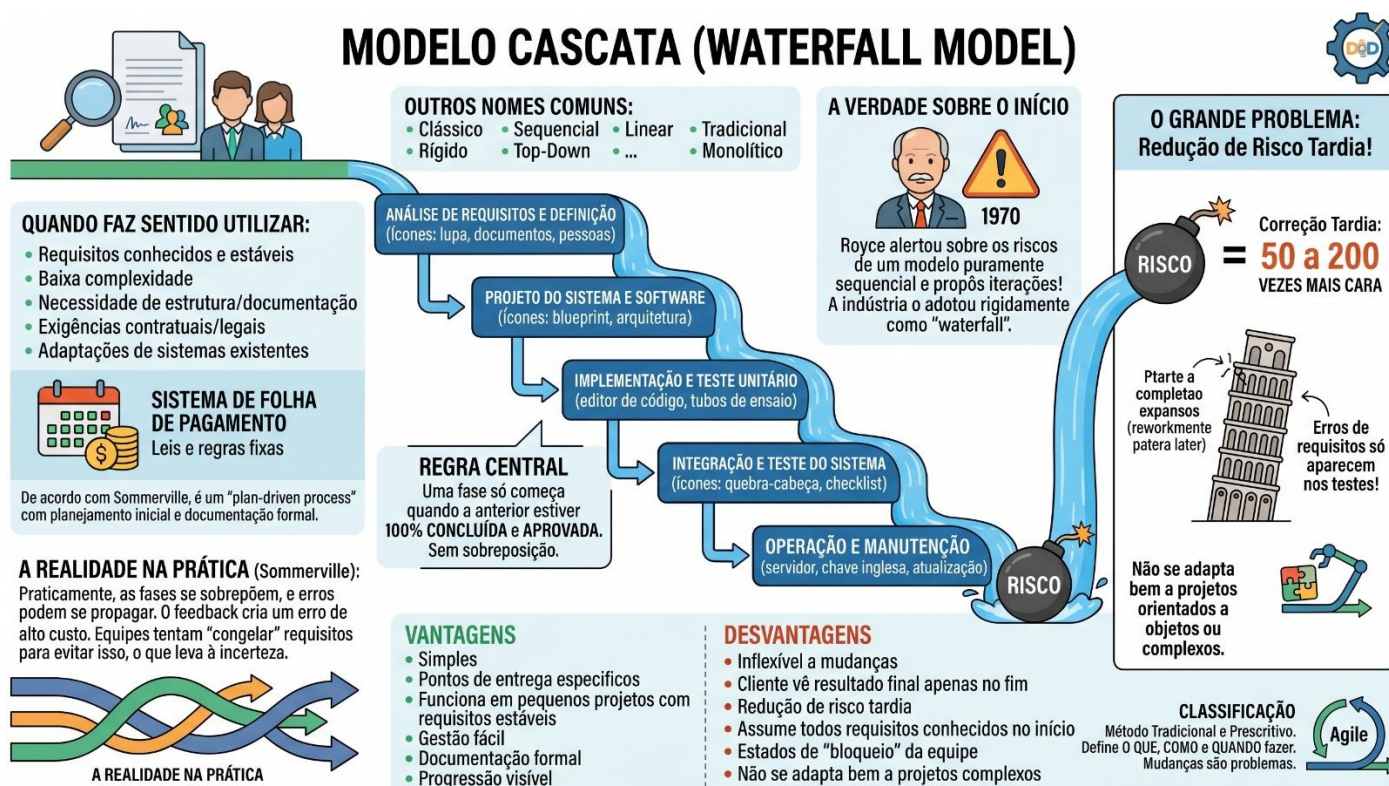
- Quando os requisitos são bem compreendidos, estáveis e pouco propensos a mudanças.
- Quando o projeto é pequeno e de baixa complexidade.
- Quando a equipe precisa de um processo altamente estruturado e documentado.
- Quando há exigências contratuais ou regulatórias que requerem documentação formal em cada fase.

- Em adaptações bem definidas de sistemas existentes.

Fora desses contextos, modelos iterativos e incrementais tendem a ser mais adequados para a realidade do desenvolvimento moderno. Vejamos agora as vantagens e desvantagens do modelo:

VANTAGENS	DESVANTAGENS
Simples de entender e fácil de aplicar.	Divisão inflexível em estágios distintos.
Fixa pontos específicos para entrega de artefatos.	Dificuldade em incorporar mudanças de requisitos.
Funciona bem em projetos pequenos com requisitos estáveis.	Cliente só vê resultados funcionais próximo ao final.
Fácil de gerenciar devido à sua estrutura rígida.	Atraza a redução de riscos – erros só aparecem nas fases finais.
Produz documentação formal e extensa em cada fase.	Apenas a fase final entrega um artefato de software operacional.
Boa aderência a outros modelos de processo de engenharia.	Pressupõe que todos os requisitos são conhecidos no início.
Torna o progresso visível e monitorável.	Pressupõe estabilidade de requisitos ao longo do tempo.
	Cria "estados de bloqueio" – membros da equipe esperam uns pelos outros.
	Não se adapta bem a projetos complexos ou orientados a objetos.

O modelo em cascata não é "para equipes fracas", mas sim mais adequado a contextos que exigem forte estruturação, documentação e previsibilidade, o que pode ajudar equipes que precisam de maior clareza no processo. Segundo Roger Pressman, trata-se de um método tradicional e altamente prescritivo, no qual tudo é definido antecipadamente – escopo, prazos, custos e requisitos – e mudanças são vistas como exceções indesejadas. Esse caráter rígido contrasta diretamente com abordagens ágeis, que tratam mudanças como algo natural e as incorporam continuamente ao desenvolvimento.



Modelo Iterativos

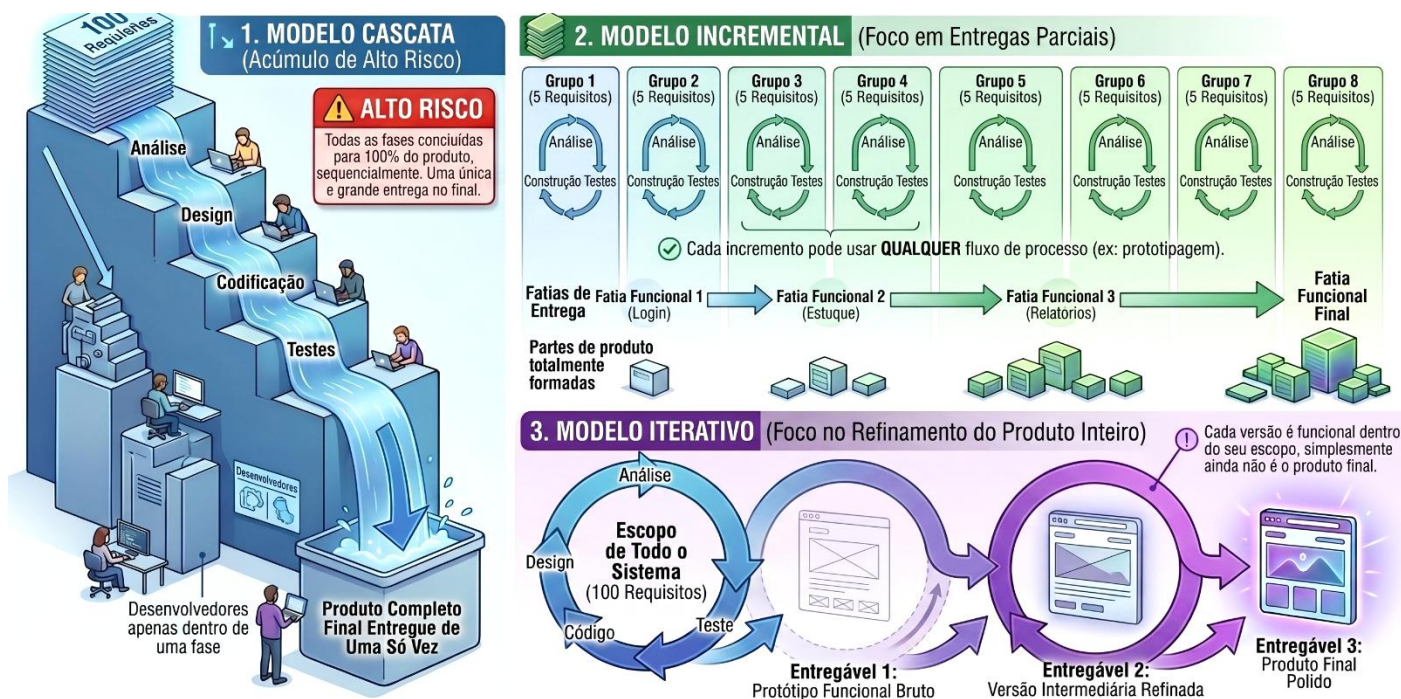
Modelos Incrementais

INCIDÊNCIA EM PROVA: ALTA

MODELO INCREMENTAL

Abordagem de desenvolvimento de software em que o sistema é construído e entregue em partes funcionais (incrementos), enquanto cada parte e o produto como um todo são continuamente refinados por meio de ciclos sucessivos (iterações). Assim, novas funcionalidades são adicionadas progressivamente, ao mesmo tempo em que melhorias, correções e ajustes são aplicados com base em feedback, permitindo entregas frequentes de valor e maior adaptação a mudanças nos requisitos.

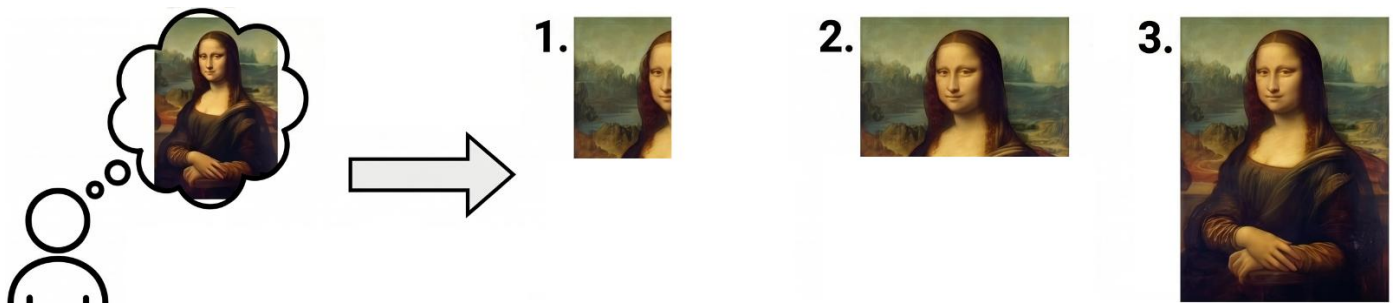
O modelo iterativo e incremental surgiu como resposta às falhas recorrentes do modelo em cascata, que acumulava riscos e levava a projetos atrasados, incompletos ou fora do esperado. Diante desse cenário, buscou-se uma abordagem que permitisse reduzir riscos mais cedo e entregar valor de forma gradual. Assim, esse modelo combina duas ideias complementares – iteração e incremento – que, embora frequentemente usadas juntas, possuem significados distintos e precisam ser compreendidas separadamente para entender seu funcionamento.



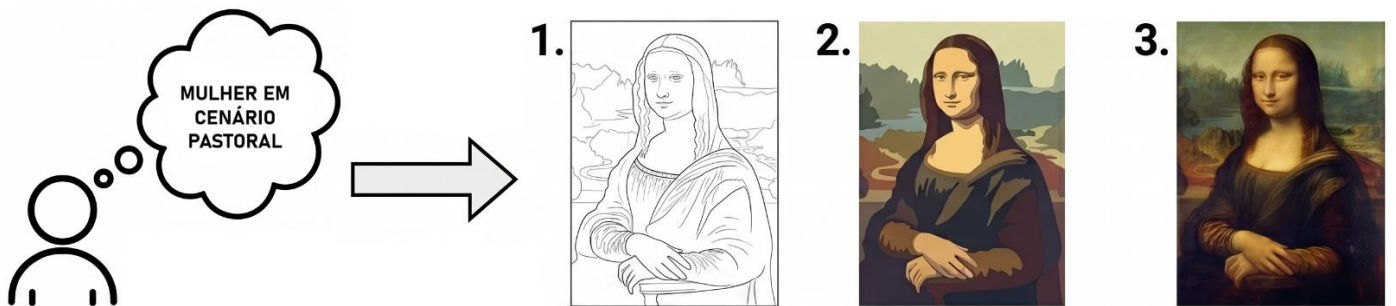
No modelo em cascata, todos os requisitos são tratados de uma só vez, de forma sequencial, sem entregas intermediárias, o que concentra riscos no final. Já no modelo incremental, os requisitos são divididos em partes menores e cada incremento entrega uma fatia funcional do sistema, independentemente do processo interno utilizado, como destaca Roger Pressman.

Por outro lado, no modelo iterativo, o sistema completo é trabalhado em ciclos sucessivos, gerando versões progressivamente mais refinadas e completas, sem entregar partes "quebradas", mas sim versões funcionais dentro de um escopo evolutivo. A analogia da Mona Lisa ilustra bem: no incremental, constrói-se por partes (rosto, fundo, mãos), enquanto no iterativo a obra inteira é refinada continuamente até atingir o resultado final.

A integração acontece ao longo do processo, não toda de uma vez no final! Cada incremento é entregue, avaliado e integrado durante o desenvolvimento – é isso que permite ao cliente dar feedback cedo.



Já o Modelo Iterativo, o esboço que evolui: imaginem um artista que começa com um esboço grosseiro do quadro inteiro. Ele não divide o quadro em partes – ele refina o quadro inteiro repetidamente, versão após versão, até chegar ao nível de detalhe e qualidade desejado. A primeira versão já é reconhecível como a Mona Lisa, mas ainda grosseira. A segunda está mais refinada. A terceira, mais ainda. Ele produz releases, isto é, versões completas mas progressivamente melhoradas do software.



Em suma: no modelo incremental, o produto cresce a cada entrega e você adiciona pedaços novos; no modelo iterativo, o produto evolui a cada ciclo e você refina o que já existe. Na prática, como destacam Roger Pressman e Ian Sommerville, os modelos modernos combinam as duas abordagens: incremental e iterativa. Isso significa que o sistema é desenvolvido em partes funcionais (incrementos), enquanto cada uma dessas partes passa por ciclos sucessivos de refinamento (iterações). Assim, o produto evolui de forma contínua, com novos incrementos sendo adicionados ao mesmo tempo em que funcionalidades já existentes são aprimoradas, justificando o uso conjunto do termo “Modelo Iterativo e Incremental”.

Cuidado com a grafia: iterativo refere-se a repetição em ciclos (iterações com entregas parciais), enquanto interativo diz respeito à interação entre partes – confundir isso é erro comum em prova. O modelo iterativo e incremental se destaca por permitir entregas desde o início, maior adaptação a mudanças e redução precoce de riscos, ao contrário do cascata. Segundo Roger Pressman, esse modelo promove feedback contínuo do cliente, facilita ajustes ao longo do desenvolvimento e melhora a qualidade do produto por meio de refinamentos sucessivos e maior possibilidade de reúso.

ASPECTOS	INCREMENTAL	ITERATIVO
IDEIA CENTRAL	Divide o produto em partes e entrega funcionalidades gradualment	Refina o produto completo por meio de ciclos contínuos sucessivos
ANALOGIA	Como montar um quebra-cabeças adicionando peças ao longo do tempo	Como um esboço que vai sendo melhorado com mais detalhes a cada ciclo
O QUE ENTREGA	Entrega builds com partes funcionais que somam valor progressivo	Entrega releases com versões aprimoradas do sistema ao longo do tempo
QUANDO INTEGRA	Integra funcionalidades ao longo do processo, não apenas no final	Integra melhorias a cada ciclo, evoluindo a versão existente do sistema

PROCESSO INTERNO	Pode usar diversos fluxos, não se limitando ao modelo cascata	Percorre fases do desenvolvimento repetidamente em ciclos iterativos
NA PRÁTICA	Geralmente aplicado junto ao iterativo para evolução incremental	Normalmente combinado ao incremental para entregas evolutivas

O modelo iterativo e incremental é superior ao cascata na maioria dos cenários modernos porque permite entregas desde o início, possibilitando que o cliente avalie o sistema continuamente e ajuste o rumo do projeto com baixo custo. Essa abordagem traz maior tolerância a mudanças, reduz riscos precocemente e melhora a qualidade por meio de refinamentos sucessivos. Segundo Ian Sommerville, o desenvolvimento incremental consiste em criar uma versão inicial, submetê-la ao feedback dos usuários e evolui-la em ciclos, intercalando especificação, desenvolvimento e validação, o que garante adaptação contínua e alinhamento com as reais necessidades do cliente.



O modelo iterativo e incremental surge como resposta às limitações do modelo em cascata, principalmente ao acúmulo de riscos e à entrega tardia de valor. Diferentemente do cascata, onde todas as fases são executadas de forma sequencial e isolada, esse modelo permite que especificação, desenvolvimento e validação ocorram de forma intercalada, com feedback contínuo ao longo do processo.

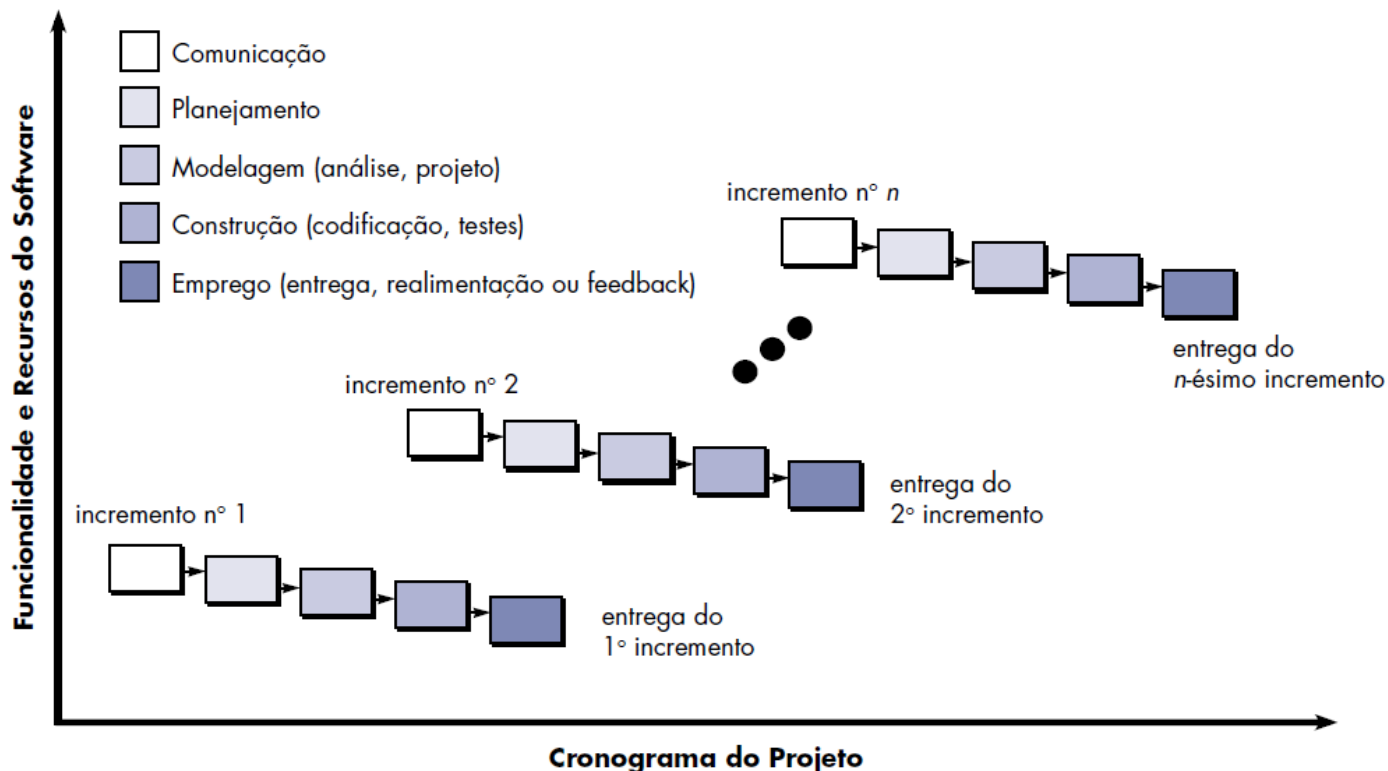
Na prática, isso significa que o software é desenvolvido em partes funcionais (incrementos), enquanto cada uma dessas partes é refinada em ciclos sucessivos (iterações). Essa combinação permite evolução contínua do produto, com entregas frequentes e melhoria progressiva da qualidade. Segundo Ian Sommerville, esse modelo reflete melhor a forma como resolvemos problemas na vida real: raramente temos uma solução completa desde o início, avançando por tentativas, ajustes e correções. Isso torna o processo mais adaptável e alinhado à realidade dos projetos de software.

Uma das principais vantagens é a redução do custo de mudanças. Como o desenvolvimento ocorre em ciclos curtos, alterações impactam apenas o incremento em andamento, evitando retrabalho massivo e tornando o processo mais eficiente. Outro benefício relevante é a facilidade de obter feedback. Clientes conseguem avaliar versões reais do sistema, o que é muito mais eficaz do que analisar documentos ou diagramas, permitindo correções antecipadas e melhor alinhamento com as necessidades reais.

Além disso, há a entrega rápida de valor. Mesmo sem o sistema completo, versões úteis já podem ser disponibilizadas ao cliente, possibilitando uso prático e geração de benefícios desde cedo. O modelo incremental pode assumir diferentes formas: dirigido a planos (incrementos definidos previamente), ágil (incrementos evoluem conforme o projeto) ou híbrido – sendo este último o mais comum na prática.

No entanto, o modelo também apresenta desvantagens. O processo pode se tornar menos visível, já que a documentação tende a ser reduzida, dificultando o controle gerencial. Além disso, mudanças contínuas podem levar à degradação arquitetural, aumentando a complexidade e o custo de manutenção ao longo do tempo. Outro desafio importante está na gestão de contratos. Como os requisitos evoluem, modelos contratuais tradicionais – baseados em escopo fixo – tornam-se difíceis de aplicar, especialmente em ambientes formais.

Por fim, conforme Roger Pressman, o modelo incremental é especialmente adequado quando os requisitos são relativamente conhecidos, mas o sistema é grande demais para ser desenvolvido de uma única vez, permitindo entregas funcionais progressivas e maior controle sobre a evolução do produto.



Segundo Roger Pressman, um incremento não deve ser confundido com um protótipo: o incremento é um produto funcional, definitivo e utilizável, enquanto o protótipo é geralmente descartável e serve para explorar incertezas. O primeiro incremento é chamado de produto essencial (core product), pois atende aos requisitos básicos do sistema, ainda sem todos os recursos complementares. Esse produto é avaliado pelo cliente e orienta o planejamento dos incrementos seguintes, que adicionam funcionalidades e refinam o sistema até sua completude.

O conceito de produto essencial é relevante para a gestão de projetos porque permite iniciar o desenvolvimento com equipes menores e expandi-las conforme o produto demonstra valor. Além disso, ajuda na mitigação de riscos técnicos, já que incrementos iniciais podem ser planejados para contornar dependências críticas, como tecnologias ainda indisponíveis.

Esse conceito evoluiu no mercado para o Minimum Viable Product, que representa uma versão mínima, porém funcional e de qualidade, utilizada para validar o produto em ambiente real com usuários. Diferentemente do protótipo, o MVP não é descartável: ele precisa ser utilizável, confiável e capaz de gerar valor. Assim, enquanto o protótipo testa ideias ou aspectos técnicos, o MVP valida o produto no contexto de negócio, sendo uma entrega real e evolutiva dentro do desenvolvimento incremental.

Modelos Evolutivos

MODELO EVOLUCIONÁRIOS

Abordagem de processo de software em que o sistema é desenvolvido por meio de versões sucessivas, cada uma delas refinando e expandindo a anterior. Em vez de especificar completamente todos os requisitos desde o início, o desenvolvimento ocorre de forma iterativa, permitindo que requisitos sejam progressivamente compreendidos e incorporados com base em feedback contínuo dos usuários. Essa abordagem reduz incertezas, melhora o alinhamento com as necessidades reais e possibilita a entrega incremental de valor, embora exija controle rigoroso para evitar perda de estrutura e qualidade ao longo das iterações.

Os modelos evolucionários surgem da constatação, destacada por Roger Pressman, de que softwares evoluem continuamente e que, em muitos projetos, não é possível definir completamente o produto desde o início. Diferentemente de um planejamento linear, esses modelos são projetados para lidar com mudanças constantes, prazos apertados e incertezas, permitindo que o sistema seja desenvolvido de forma progressiva e adaptativa.

Embora sejam uma categoria específica, os modelos evolucionários são considerados iterativos, pois também trabalham em ciclos sucessivos que produzem versões cada vez mais completas do software. A diferença não está na repetição em si, mas no propósito desses ciclos. A principal distinção em relação ao modelo incremental está na natureza dos requisitos. No incremental, os requisitos já são razoavelmente conhecidos – o desafio é apenas dividir e entregar o sistema em partes. Já no evolucionário, os requisitos são incertos ou incompletos, e o próprio processo serve para descobri-los e refiná-los ao longo do desenvolvimento.

Isso muda completamente a lógica do projeto: no incremental, você sabe o que construir, mas não consegue fazer tudo de uma vez; no evolucionário, você ainda está entendendo o que precisa ser construído. A segunda diferença fundamental está no tipo de entrega em cada iteração. No modelo incremental, cada ciclo precisa gerar funcionalidade operacional utilizável pelo cliente. Já no evolucionário, especialmente no início, as iterações podem produzir apenas protótipos, modelos ou estudos – sem entregar software funcional.

Assim, o modelo evolucionário permite explorar alternativas, validar hipóteses e reduzir incertezas antes de comprometer esforços com implementação definitiva. Em síntese, os modelos evolucionários são iterativos voltados à descoberta e refinamento de requisitos, enquanto os incrementais são iterativos voltados à entrega progressiva de funcionalidades já conhecidas.

ASPECTOS	INCREMENTAL	EVOLUCIONÁRIO
REQUISITOS	Conhecidos desde o início	Incertos, descobertos ao longo do processo
ENTREGA POR ITERAÇÃO	Sempre funcionalidade operacional	Pode ser documentação, modelo ou protótipo
CONTEXTO IDEAL	Escopo grande, requisitos estáveis	Alta incerteza, mudanças esperadas
OBJETIVO	Implementar o que já se sabe	Descobrir e refinar o que se precisa construir

E como Sommerville trata essa distinção? Aqui vale uma observação importante sobre a evolução do pensamento de Sommerville. Nas edições mais recentes do seu livro, ele reorganizou conceitualmente a classificação dos modelos de processo, passando a estruturá-los em torno de duas grandes filosofias:

- **Processos Dirigidos a Planos (Plan-Driven):** onde as atividades são planejadas com antecedência e o progresso é medido em relação a esse plano.

- **Processos Ágeis:** onde o planejamento é incremental e é mais fácil mudar o processo para refletir as necessidades mutáveis do cliente.

Dentro dessa nova organização, o desenvolvimento incremental absorveu muito do que antes era chamado de evolucionário. Essa não foi uma decisão arbitrária – foi uma reorganização conceitual deliberada de Sommerville, que considerou que a distinção terminológica entre incremental e evolucionário gerava mais confusão do que clareza.

O resultado prático para vocês: nas questões que se baseiam em Sommerville, é comum que evolucionário e incremental sejam tratados como sinônimos ou como parte de uma mesma família. Nas questões que se baseiam em Pressman, a distinção entre os dois pode ser cobrada – e as duas dimensões que vimos acima são os critérios que ele usa.

Agora que entendemos o conceito geral, é hora de saber quais modelos se enquadram nessa categoria. Para Pressman, os principais modelos evolucionários são:

- **Prototipação:** focado na construção de protótipos – versões preliminares e exploratórias do sistema – para ajudar a descobrir e refinar requisitos. É o modelo ideal quando o cliente sabe o que quer, mas não consegue descrever com precisão como deve ser. O protótipo serve como ferramenta de comunicação e descoberta.
- **Modelo Espiral:** combina a natureza iterativa da prototipação com os aspectos sistemáticos e controlados do modelo em cascata. Seu diferencial é o gerenciamento explícito de riscos a cada ciclo – cada volta da espiral inclui uma fase dedicada a identificar, analisar e mitigar riscos antes de avançar. É o modelo mais adequado para projetos grandes, complexos e de alto risco.

Veremos cada um desses modelos em detalhes nas próximas seções. Por ora, o importante é entender que ambos compartilham a mesma característica central dos modelos evolucionários: lidam com incerteza e mudança de forma estruturada, em contraste com o modelo incremental, que pressupõe requisitos razoavelmente estáveis.



COMPARAÇÃO: MODELO INCREMENTAL vs. MODELO EVOLUCIONÁRIO		
ASPECTO	INCREMENTAL	EVOLUCIONÁRIO
Requisitos	Conhecidos desde o início	Incertos, descobertos ao longo do processo
Entregável por Iteração	Sempre Software Operacional	Pode ser Protótipo, Modelo, Documentação ou Maquete
Contexto Ideal	Grande escopo, requisitos estáveis	Alta incerteza, mudanças esperadas
Objetivo Principal	Entregar o que já é conhecido	Descobrir e refinar o que precisa ser construído



Prototipação

INCIDÊNCIA EM PROVA: MÉDIA

PROTOTIPAÇÃO

Abordagem em que uma versão inicial e simplificada do sistema, chamada protótipo, é construída para ajudar na compreensão e validação dos requisitos. Esse protótipo é utilizado pelos usuários para explorar funcionalidades, esclarecer expectativas e fornecer feedback, permitindo que requisitos sejam refinados antes do desenvolvimento definitivo. O protótipo pode ser descartado após cumprir seu papel ou evoluir para o sistema final, dependendo da estratégia adotada, sendo especialmente útil em contextos de alta incerteza ou dificuldade de comunicação entre cliente e equipe técnica.

A prototipação surge para resolver um problema clássico: a falta de clareza nos requisitos. Quando o cliente não consegue explicar exatamente o que precisa, ou quando há incertezas técnicas relevantes, seguir um modelo rígido como o cascata tende ao fracasso. Nesse cenário, a prototipação permite construir algo tangível desde cedo, ajudando a explorar ideias, reduzir dúvidas e orientar melhor o desenvolvimento.

Segundo Ian Sommerville, um protótipo é uma versão inicial do sistema criada para demonstrar conceitos, testar alternativas de projeto e compreender melhor o problema. O foco não está em entregar um produto final, mas sim em aprender com essa versão preliminar. Ele funciona como um instrumento de experimentação, permitindo validar hipóteses e reduzir incertezas antes de decisões definitivas.

Roger Pressman destaca que a prototipação pode ser usada de duas formas: como um modelo de processo completo ou como uma técnica dentro de outros modelos. Na prática, o uso mais comum é

como apoio a outros processos, especialmente em contextos iterativos e ágeis, onde o protótipo ajuda a esclarecer requisitos antes da implementação definitiva do sistema.

Na engenharia de requisitos, o protótipo facilita a elicitac o e a valida o, pois usu rios conseguem avaliar algo concreto em vez de documentos abstratos. Isso permite identificar erros, lacunas e excessos com mais facilidade, melhorando a comunica o entre cliente e equipe. A intera o com o prot tipo torna o processo mais eficiente e reduz significativamente o risco de interpreta es equivocadas.

No projeto do sistema, a prototipa o tamb m   valiosa para testar solu es t cnicas antes de decis es definitivas.   poss vel validar algoritmos, estruturas de dados ou abordagens arquiteturais sem comprometer todo o sistema. Isso reduz riscos t cnicos e evita escolhas inadequadas que poderiam gerar alto custo de corre o nas fases posteriores do desenvolvimento.

Al m disso, a prototipa o   essencial no desenvolvimento de interfaces gr ficas. Descri es textuais e diagramas n o conseguem representar adequadamente o comportamento din mico de uma interface. A intera o direta com vers es visuais permite avaliar usabilidade, fluxo e experi ncia do usu rio. Por isso, criar prot tipos   a forma mais eficaz de projetar interfaces realmente adequadas.

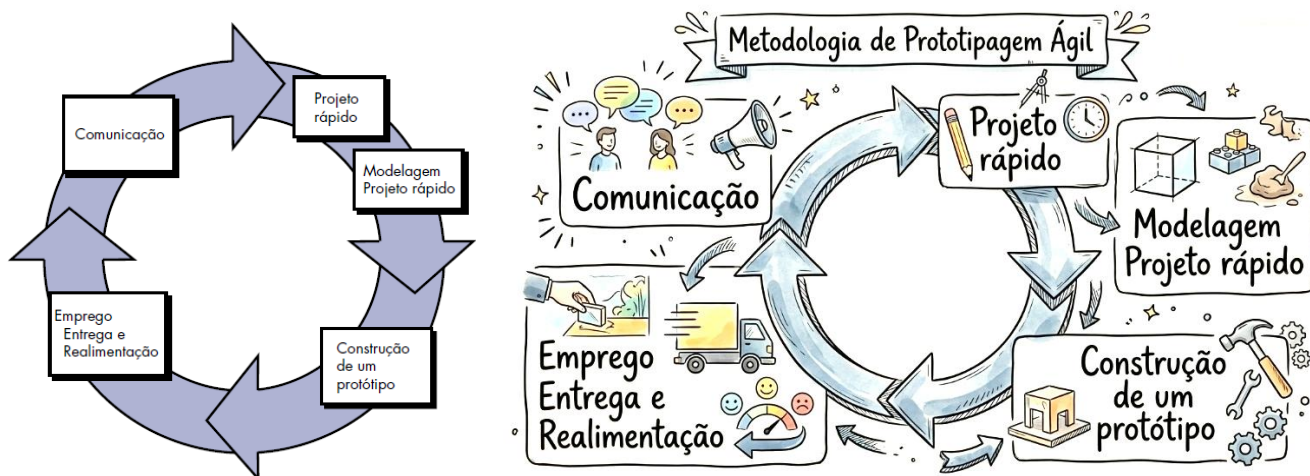
Nesse sentido, a prototipa o   uma abordagem voltada ao aprendizado e   redu o de incertezas. Ela permite explorar solu es, obter feedback cont nuo e tomar decis es mais seguras ao longo do desenvolvimento. Ao inv s de tentar prever tudo no in cio, o processo passa a evoluir com base em experimenta o, tornando-se mais adapt vel e alinhado  s necessidades reais do usu rio.

Ian Sommerville descreve um modelo de processo para desenvolvimento de prot tipos com quatro etapas bem definidas:

ETAPAS	DESCRI�O	ARTEFATO GERADO
ESTABELECEER OBJETIVOS DO PROT�TIPO	Define <i>por que</i> o prot�tipo ser� constru�do: validar requisitos, explorar interface ou testar viabilidade t�cnica.	Plano de prototipa�o
DEFINIR FUNCIONALIDADE DO PROT�TIPO	Decide o <i>que</i> o prot�tipo vai incluir, priorizando funcionalidades de maior risco ou incerteza.	Defini�o geral
DESENVOLVER PROT�TIPO	Constru�o r�pida do prot�tipo, geralmente com simplifica�es e ferramentas de desenvolvimento �gil.	Prot�tipo execut�vel
AVALIAR O PROT�TIPO	Usu�rios e stakeholders testam e fornecem feedback para guiar a pr�xima itera�o ou o desenvolvimento final.	Relat�rio de avalia�o



Pressman descreve o processo de prototipação de forma ligeiramente diferente, mas complementar. Para ele, o paradigma começa com a comunicação: uma reunião com todos os envolvidos para definir os objetivos gerais do software, identificar quais requisitos já são conhecidos e mapear quais áreas precisam de exploração mais aprofundada.



Em seguida, ocorre um planejamento rápido, seguido de modelagem na forma de um "projeto rápido". Esse projeto rápido se concentra nos aspectos do software que serão visíveis aos usuários finais – como o layout da interface ou os formatos de exibição de dados. Não é um projeto completo e definitivo; é uma representação suficiente para construir o protótipo. O projeto rápido leva à construção do protótipo, que é empregado e avaliado pelos envolvidos.

O feedback coletado serve para aprimorar os requisitos, e o ciclo se repete – ajustando o protótipo às necessidades dos interessados até que os requisitos estejam bem compreendidos. E o que fazer com o protótipo depois? Essa é uma questão central – e a resposta depende do tipo de prototipação adotado. Pressman e Sommerville reconhecem dois destinos possíveis para um protótipo:

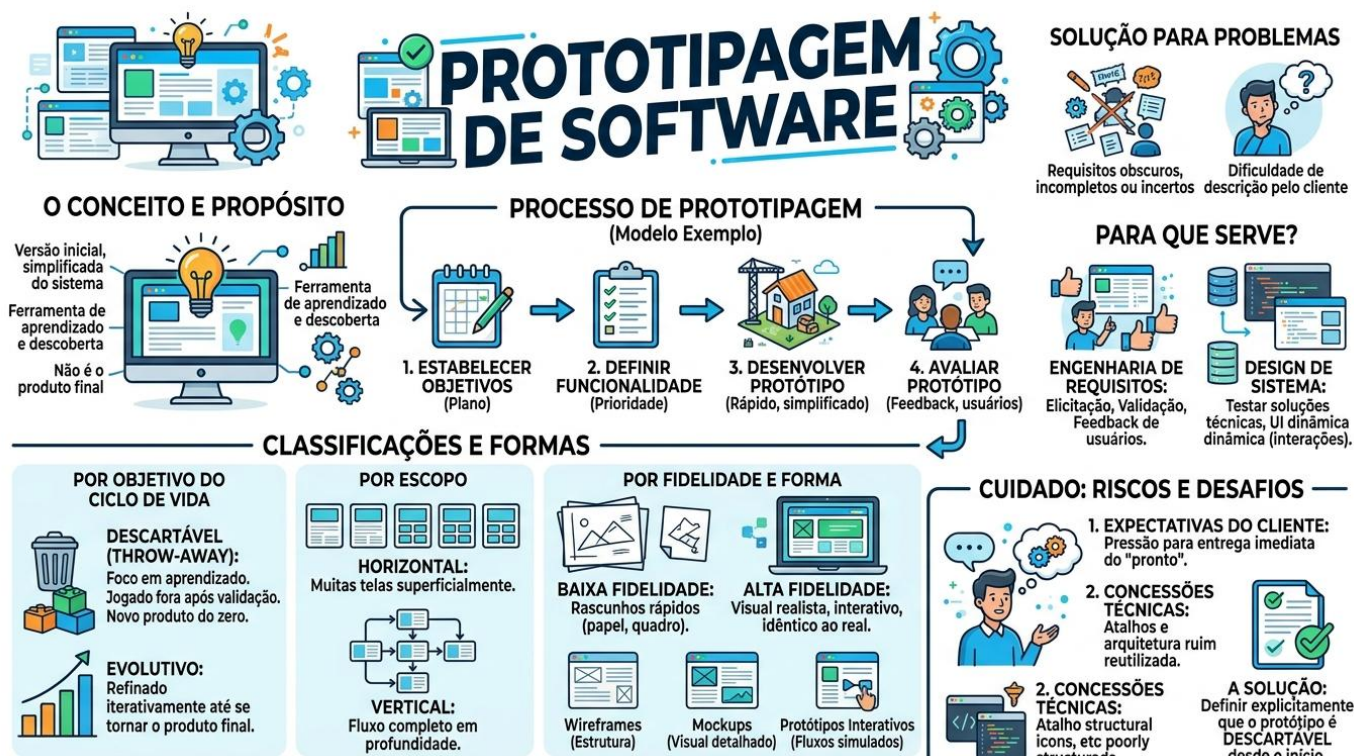
TIPOS	DESCRIÇÃO
PROTÓTIPO DESCARTÁVEL	O protótipo cumpre sua função de explorar e clarificar requisitos e depois é jogado fora. O produto final é desenvolvido do zero, com base no aprendizado obtido. Pressman usa uma analogia direta: o protótipo é "o primeiro sistema" – que deve ser descartado para dar lugar a uma versão redesenhada e bem estruturada.

PROTÓTIPO EVOLUCIONÁRIO

O protótipo não é descartado. Pelo contrário, ele é refinado iterativamente (repetidamente) até evoluir para o sistema final. Esse tipo de prototipação é mais arriscado, pois o código desenvolvido rapidamente e sem rigor adequado pode acabar virando o produto entregue ao cliente.

Em suma, a prototipação se baseia na construção de um protótipo que auxilia a levantar os requisitos junto aos usuários e depois ser descartado; ou ser refinado progressivamente até se tornar o sistema final. Sommerville formaliza essa distinção em dois tipos específicos – e essa nomenclatura pode cair em prova: Desenvolvimento Exploratório ou Prototipação Descartável.

TIPOS	DESCRIÇÃO
DESENVOLVIMENTO EXPLORATÓRIO (EVOLUCIONÁRIO)	O objetivo é trabalhar com o cliente para explorar os requisitos e entregar um sistema final. O desenvolvimento começa com as partes do sistema já compreendidas, e o sistema evolui por meio da adição de novas características propostas pelo cliente ao longo do processo. O protótipo não é descartado – ele se torna o produto.
DESENVOLVIMENTO THROWAWAY (DESCARTÁVEL)	O objetivo é compreender os requisitos do cliente e desenvolver uma definição melhor para o sistema. O protótipo se concentra na experimentação dos requisitos mal compreendidos, mas é descartado ao final do processo de descoberta. O sistema real é desenvolvido separadamente.



Segundo Ian Sommerville, quando o termo "prototipação" é usado sem qualificação, refere-se à prototipação descartável, ou seja, um processo iterativo voltado à construção de um sistema experimental que não será entregue ao cliente. Já o desenvolvimento evolucionário é tratado como incremental. Em provas, portanto, prototipação tende a significar versão descartável.

Roger Pressman aponta como primeiro risco a criação de expectativas irreais pelo cliente. Ao ver o protótipo funcionando, ele assume que o produto final está próximo, ignorando atalhos técnicos usados. Isso gera pressão por entrega rápida de algo que ainda precisaria ser reconstruído com qualidade adequada.

Outro problema é que o desenvolvedor pode adotar soluções técnicas inadequadas para acelerar o protótipo, como arquitetura improvisada ou código ineficiente. Depois, pressionado, acaba reutilizando esse código no produto final, o que compromete qualidade, manutenção e evolução do sistema ao longo do tempo.

Sommerville ainda destaca que protótipos descartáveis frequentemente não são descartados na prática. Por pressão de custo e prazo, são levados à produção, resultando em sistemas frágeis e com dívida técnica. A solução é definir claramente, desde o início, que o protótipo será descartado, com alinhamento entre cliente, equipe e gestão.

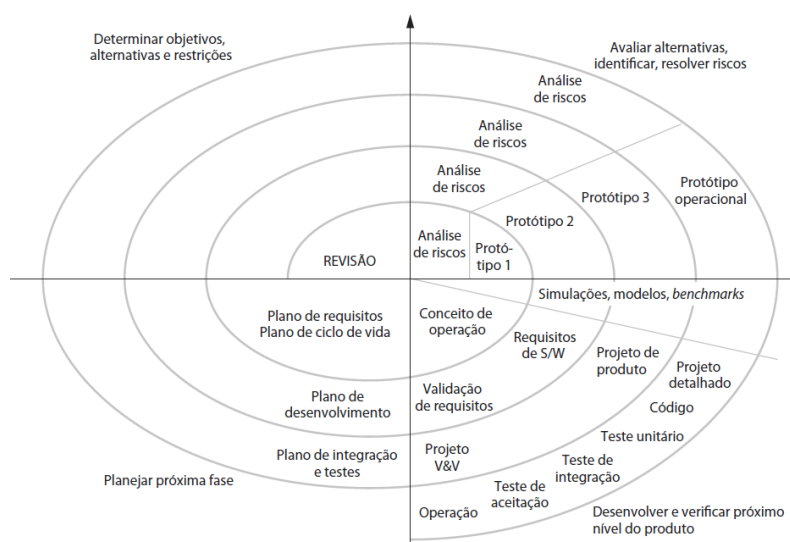
Modelo em Espiral

INCIDÊNCIA EM PROVA: MÉDIA

MODELO EM ESPIRAL

Processo de desenvolvimento evolutivo orientado a riscos, no qual o software é construído por meio de ciclos iterativos (espirais). Cada ciclo envolve atividades de planejamento, análise de riscos, engenharia e avaliação com o cliente, permitindo identificar e mitigar incertezas antes de avançar. A cada volta da espiral, o sistema é progressivamente refinado e ampliado, combinando características de prototipação e desenvolvimento incremental, com forte ênfase na gestão sistemática de riscos ao longo de todo o projeto.

O Modelo Espiral, proposto por Barry Boehm em 1988, surgiu para resolver uma lacuna dos modelos anteriores: a ausência de um tratamento sistemático dos riscos. Sua principal inovação é colocar a análise de riscos como elemento central e obrigatório em cada ciclo do desenvolvimento, tornando o processo mais seguro e controlado.

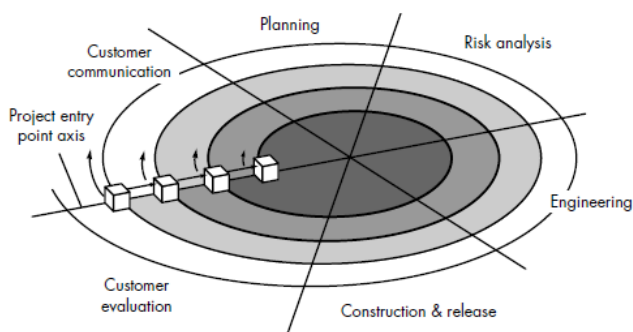


Nos modelos anteriores, os riscos eram tratados de forma limitada: no Cascata, surgiam tarde; no Incremental, eram reduzidos indiretamente; e na Prototipação, tratados de forma pontual. O Espiral integra a identificação, análise e mitigação de riscos em todas as etapas, de forma contínua e estruturada. Em vez de uma sequência linear, o modelo é representado como uma espiral que se expande a cada ciclo. Cada volta corresponde a uma fase do desenvolvimento, indo de aspectos mais abstratos, como viabilidade e requisitos, até etapas mais concretas, como implementação e testes.

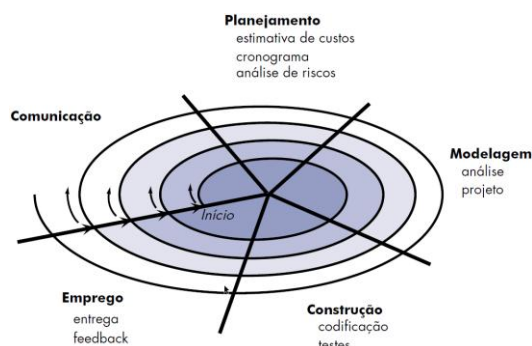
Assim, o modelo pode cobrir todo o ciclo de vida do software, desde a concepção até a manutenção e evolução. Diferente de outros modelos, ele não termina na entrega, podendo ser aplicado continuamente ao longo da vida útil do sistema, com adaptação progressiva.

SETORES	O QUE ACONTECE AQUI?
DETERMINAR OBJETIVOS, ALTERNATIVAS E RESTRIÇÕES	Definem-se os objetivos da fase atual (melhorar performance, corrigir funcionalidade, garantir segurança, etc.), identificam-se alternativas disponíveis (reúso de componentes, comprar um produto pronto, desenvolver do zero) e mapeiam-se as restrições impostas (custo, prazo, recursos humanos).
AVALIAR ALTERNATIVAS, IDENTIFICAR E RESOLVER RISCOS	As alternativas são avaliadas em relação aos objetivos e restrições. Áreas de incerteza são identificadas – custos excessivos, lacunas técnicas, requisitos ambíguos – e os riscos são resolvidos ou reduzidos antes de avançar. Um protótipo pode ser construído aqui para explorar incertezas e validar hipóteses.
DESENVOLVER E TESTAR	Após a redução dos riscos, seleciona-se um modelo de desenvolvimento adequado para aquela fase específica – pode ser prototipação evolucionária, Cascata, modelo baseado em componentes, entre outros. O software é desenvolvido e testado.
PLANEJAR PRÓXIMAS FASES	Revisa-se o trabalho realizado e decide-se sobre o prosseguimento ao próximo loop. Se a decisão for continuar, elaboram-se os planos para a próxima fase. Aqui ocorre um marco de ancoragem – que veremos em detalhes a seguir.

Uma observação importante: os setores são fixos para todos os loops, o que muda a cada volta não são os setores, mas o conteúdo e o nível de detalhe trabalhado dentro deles. Roger Pressman, a partir da 6ª edição do seu livro, propõe uma organização ligeiramente diferente, com cinco setores: Comunicação, Planejamento, Modelagem, Construção e Implantação.



PRESSMAN - 5ª EDIÇÃO



PRESSMAN - 7ª EDIÇÃO

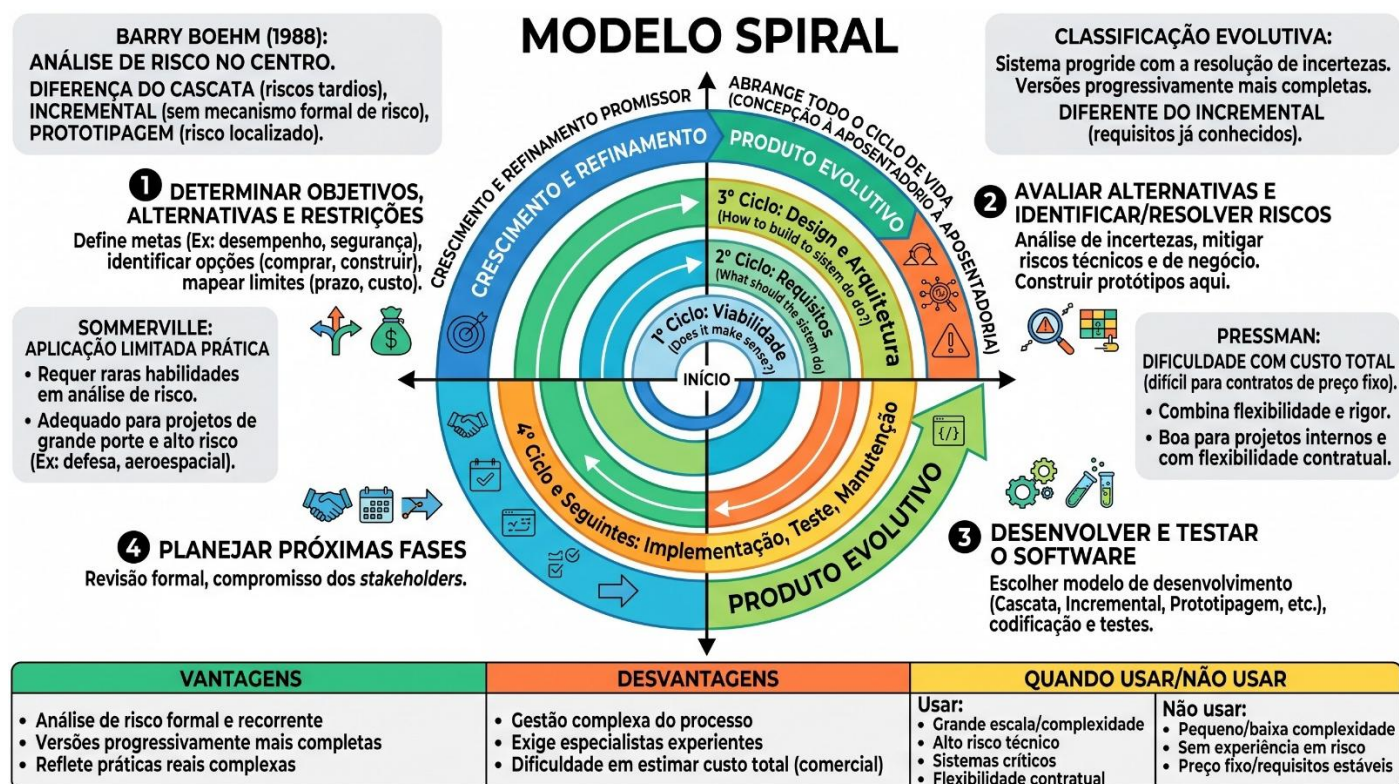
SETORES (R. PRESSMAN)	DESCRIÇÃO
COMUNICAÇÃO	Levantamento de necessidades e alinhamento com os envolvidos.
PLANEJAMENTO	Estimativa de custos, cronograma e análise de riscos.
MODELAGEM	Análise e design do sistema.
CONSTRUÇÃO	Codificação e testes.
IMPLANTAÇÃO	Entrega ao cliente e coleta de feedback.

As versões do Modelo Espiral variam entre autores, com divisões de três a seis setores. Por isso, em provas, deve-se adotar como referência a versão original de Barry Boehm, com quatro setores. Essas variações geram nomes diferentes, mas não alteram a essência do modelo, que permanece centrada na evolução progressiva orientada por riscos.

Ao final de cada ciclo, há uma decisão formal sobre continuar ou encerrar o projeto, registrada nos chamados marcos de ancoragem. Nesses pontos, cliente, equipe e gerência avaliam os resultados, validam entregas e definem os próximos passos. Esses marcos garantem controle, transparência e auditabilidade, evitando que o projeto avance sem validação consistente.

O Espiral é classificado como evolucionário porque foi projetado para ambientes de alta incerteza, onde o produto evolui conforme os riscos são reduzidos. Nas primeiras iterações, podem existir apenas modelos ou protótipos; com o tempo, surgem versões mais completas. Assim, requisitos e solução são descobertos progressivamente ao longo do processo.

Segundo Roger Pressman, o Espiral combina a flexibilidade iterativa da prototipação com o rigor do Cascata, tendo a análise de riscos como elemento central. Diferente do incremental, ele não parte de requisitos bem definidos, mas evolui junto com a compreensão do problema e das soluções possíveis. Para Ian Sommerville, o modelo é conceitualmente robusto, porém pouco utilizado na prática. Ele é mais adequado a projetos grandes e críticos, como sistemas de defesa ou infraestrutura, pois exige análise de riscos rigorosa, algo que poucas equipes dominam com maturidade suficiente.



Outra limitação relevante, apontada por Pressman, é a dificuldade de estimar custos totais. Como o escopo evolui a cada ciclo, o modelo não se adapta bem a contratos de preço fixo, comuns no mercado. Por isso, é mais indicado para projetos internos, de grande porte ou com alta flexibilidade contratual e equipes experientes.

VANTAGENS	DESVANTAGENS
Análise de riscos sistemática e formal a cada ciclo.	Exige analistas de risco experientes e altamente qualificados.
Versões progressivamente mais completas e refinadas a cada loop.	Exige equipe de desenvolvimento com perfil técnico elevado.
Pode ser aplicado a todo o ciclo de vida do software.	Gerenciamento de processo mais complexo do que outros modelos.
Combina o rigor do Cascata com a flexibilidade do iterativo.	Não recomendado para projetos pequenos e simples.
Reflete as práticas reais de engenharia para projetos complexos.	Difícil de justificar em contratos de preço fixo.

Estimativas realistas por considerar riscos formalmente.	Baixa adoção comercial pela exigência de competências avançadas.
Marcos de ancoragem garantem comprometimento formal dos envolvidos.	

Recomenda-se utilizar o Modelo Espiral quando o projeto é de grande escala e alta complexidade; há alto risco técnico (tecnologias novas, requisitos críticos de desempenho ou segurança); o sistema é crítico (defesa, aeroespacial, saúde, infraestrutura financeira); a equipe possui experiência sólida em análise formal de riscos; e há flexibilidade contratual (o cliente aceita escopo evolutivo).

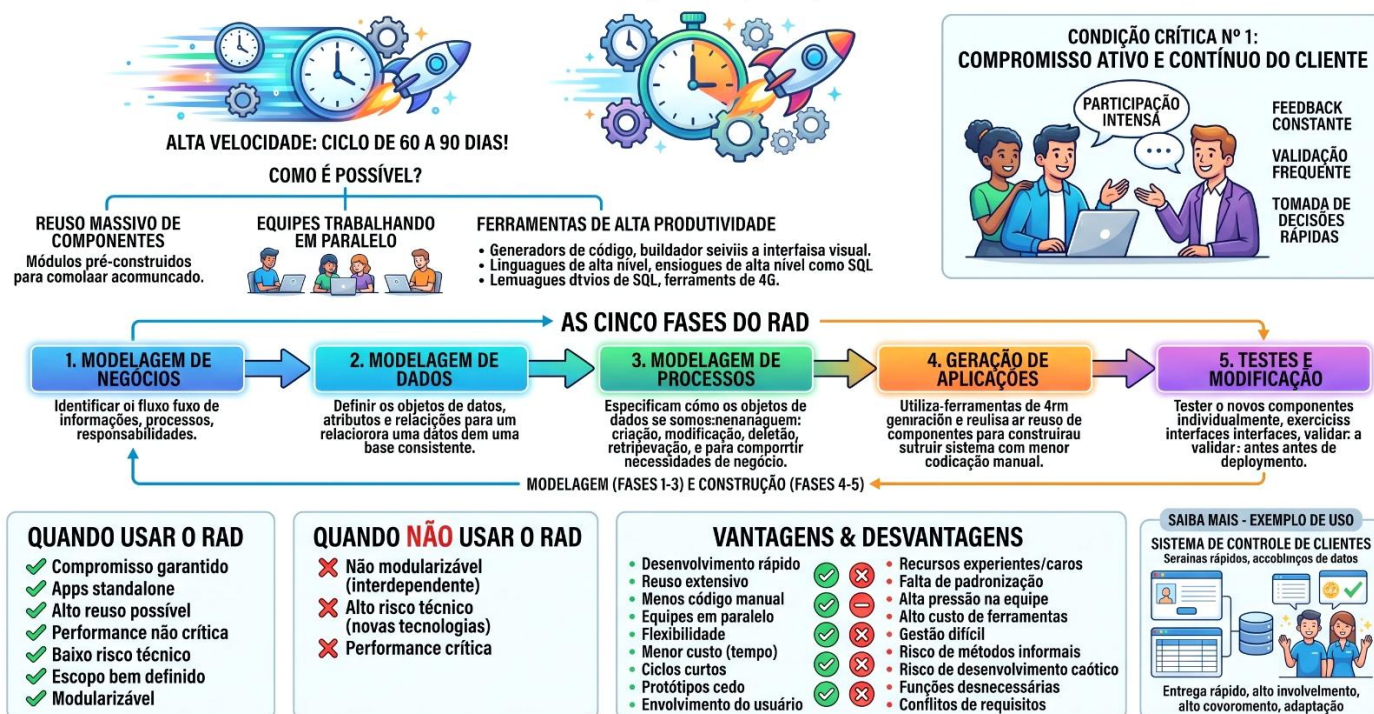
Por outro lado, não use o Modelo Espiral quando: o projeto é pequeno ou de baixa complexidade; a equipe não tem experiência em gestão de riscos; o contrato exige preço fixo e escopo predefinido; os requisitos são estáveis e bem conhecidos desde o início. Nesses casos, o Incremental ou até o Modelo em Cascata seriam mais simples e eficientes.

Modelo Especializados

RAD (RAPID APPLICATION DEVELOPMENT)

Modelo de desenvolvimento de software que prioriza a rapidez na entrega por meio da construção iterativa de protótipos e do uso intensivo de ferramentas de alto nível, como geradores de código e construtores visuais de interfaces. Ele envolve fortemente os usuários no processo, permitindo validações constantes e ajustes rápidos, reduzindo a necessidade de planejamento detalhado inicial e focando na adaptação contínua até atingir os requisitos desejados.

DESENVOLVIMENTO RÁPIDO DE APLICAÇÕES (RAD) - O MODELO DE ALTA VELOCIDADE



ENGENHARIA DE SOFTWARE BASEADA EM COMPONENTES

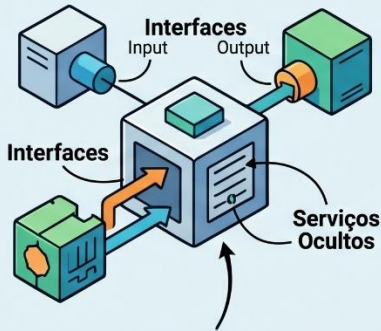
Abordagem de desenvolvimento em que o sistema é construído a partir da seleção, adaptação e integração de componentes reutilizáveis, previamente desenvolvidos e testados. Em vez de desenvolver todo o software do zero, a ênfase está no reuso de módulos existentes (como bibliotecas, frameworks ou sistemas COTS), o que

reduz tempo, custo e riscos. O processo envolve identificar requisitos, buscar componentes adequados, avaliá-los, adaptar o sistema a eles quando necessário e integrar tudo em uma arquitetura coerente, exigindo atenção à compatibilidade, qualidade e evolução dos componentes.

ENGENHARIA DE SOFTWARE BASEADA EM COMPONENTES (CBSE)

Uma abordagem focada no reúso de componentes existentes e testados para construir sistemas, reduzindo tempo, custo e risco.

O QUE É UM COMPONENTE?



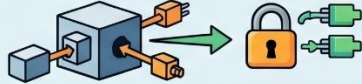
COMPONENTE?

Um bloco de construção modular: **executável, independente, padronizado, reutilizável**. Encapsula sua implementação interna e expõe interfaces bem definidas.

Como uma peça que se encaixa, você conhece seus **serviços** mas **não sua lógica interna**.

FUNDAMENTOS TÉCNICOS

OCP - PRINCÍPIO ABERTO-FECHADO
Aberto para extensão, mas fechado para modificação. Permite novas funcionalidades sem alterar o código original testado.

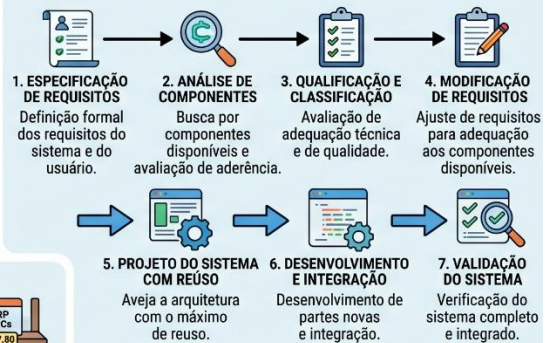


COTS (COMMERCIAL OFF-THE-SHELF)

Produtos de software comerciais prontos para uso (frameworks, bibliotecas, ERPs) com interfaces definidas, desenvolvidos por fornecedores.



ETAPAS DO PROCESSO (BASEADO EM PRESSMAN)



Ponto Crucial: Custos de **INTEGRAÇÃO** e **TESTE** são **SIGNIFICATIVOS** e frequentemente subestimados!

NATUREZA E PERSPECTIVAS

MODELO EVOLUCIONÁRIO E ITERATIVO (como Espiral/Incremental).
Restrição Estrutural: COTS não modificáveis.



PERSPECTIVA DE SOMMERVILLE (Integração e Configuração)

3 TÍPICOS CENÁRIOS DE REÚSO:



MÉTODOS FORMAIS

Abordagem de desenvolvimento de software que utiliza especificações baseadas em notações matemáticas rigorosas para descrever, projetar e verificar sistemas. Nessa abordagem, os requisitos são expressos de forma precisa e sem ambiguidades, permitindo a aplicação de técnicas de prova e verificação para demonstrar propriedades do sistema, como correção e consistência. O objetivo é reduzir erros desde as fases iniciais e aumentar a confiabilidade do software, sendo especialmente útil em sistemas críticos, embora exija alto custo, conhecimento especializado e maior esforço de desenvolvimento.

MÉTODOS FORMAIS NO DESENVOLVIMENTO DE SOFTWARE



O QUE SÃO?

Uso de notação matemática rigorosa para especificar, projetar e verificar sistemas. Requisitos precisos e sem ambiguidades. Permite provas de propriedades (corretude, consistência).

QUANDO USAR?

Sistemas complexos com requisitos rigorosos de corretude e confiabilidade. Falhas podem causar perda de vidas ou sérios danos econômicos.



FASES PRINCIPAIS DO PROCESSO FORMAL



NOTA CONCEITUAL: SEM PROTÓTIPOS

Validação é por prova matemática, não nos protótipos funcionais para avaliação do usuário.

DIFERENTES USOS



VANTAGENS

- Precisão e Ausência de Ambiguidade
- Detecção Precoce de Problemas nos Requisitos
- Verificação Rigorosa (identifica erros ocultos)
- Promessa de Software Livre de Defeitos (demonstra ausência de classes de erros)

QUESTÕES COMENTADAS

1. (FGV / Câmara Municipal de Fortaleza - 2024) O modelo de desenvolvimento de software que possui ênfase na documentação extensiva e planejamento detalhado que é caracterizado por uma abordagem sequencial e linear, no qual cada fase deve ser concluída antes que a próxima seja iniciada, é conhecido como

- a) Modelo Espiral.
- b) Desenvolvimento Orientado a Testes (TDD).
- c) Modelo Lean.
- d) Modelo em Cascata.
- e) Desenvolvimento Ágil.

Comentários:

(a) Errado. O modelo Espiral é iterativo e orientado a riscos, não segue uma sequência linear rígida com fases totalmente fechadas.

(b) Errado. TDD é uma prática de desenvolvimento focada em testes antes do código, não um modelo sequencial com documentação extensiva.

(c) Errado. O modelo Lean prioriza eficiência, redução de desperdícios e adaptação, destoando do planejamento detalhado e linear.

(d) Correto. O modelo em Cascata é sequencial e linear, com forte ênfase em planejamento detalhado e documentação em cada fase.

(e) Errado. O desenvolvimento ágil valoriza adaptação contínua e entregas incrementais, não uma execução rígida fase a fase.

Gabarito: Letra D

2. (FGV / Prefeitura de Cuiabá - 2024) Entre as metodologias prescritivas de desenvolvimento de software, a metodologia em Cascata subdivide o desenvolvimento de um novo software em cinco atividades: Engenharia de Sistemas, Projeto, Codificação, Teste e Manutenção. Analise as afirmativas a seguir relativas à metodologia Cascata.

I - A interação com o cliente ocorre somente nas fases de Projeto e Manutenção.

II - As atividades de Codificação e Testes são executadas em paralelo para reduzir o tempo de desenvolvimento do software.

III - Os requisitos do sistema são determinados na atividade de Engenharia de sistemas.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) III, apenas.
- d) I e II, apenas.
- e) II e III, apenas.

Comentários:

(I) Errado. Na metodologia em Cascata, o contato com o cliente não fica restrito apenas a Projeto e Manutenção, pois há definição e alinhamento também em outros momentos do processo.

(II) Errado. No modelo Cascata, as fases seguem de forma sequencial, sem paralelismo entre Codificação e Testes como regra para acelerar o desenvolvimento.

(III) Correto. A atividade de Engenharia de Sistemas contempla o levantamento e a determinação dos requisitos que orientarão as etapas seguintes.

Gabarito: Letra C

3. (FGV / TJ SE - 2023) O gerente de TI de sua empresa solicitou a implementação de um pequeno sistema para geração da folha salarial. Como será um sistema simples, com regras imutáveis e bem definidas, o analista já definiu os requisitos e efetuou a modelagem do sistema. Após o final da implementação, será iniciada a fase de testes e, por fim, a implantação no setor financeiro. De acordo com esse contexto, o modelo de ciclo de vida adotado foi:

- a) RUP;
- b) modelo incremental;
- c) prototipagem;
- d) modelo evolutivo;
- e) modelo em cascata.

Comentários:

- (a) Errado. O RUP é iterativo e incremental, com ciclos e refinamentos sucessivos, diferente da sequência linear descrita no enunciado.
- (b) Errado. No modelo incremental, o sistema é entregue em partes sucessivas, o que não aparece no caso apresentado.
- (c) Errado. A prototipagem foca na construção de protótipos para validar requisitos, mas aqui os requisitos já estão definidos desde o início.
- (d) Errado. O modelo evolutivo prevê evolução contínua do sistema ao longo das entregas, o que não condiz com o fluxo único descrito.
- (e) Correto. O modelo em cascata segue etapas sequenciais e bem definidas, como requisitos, modelagem, implementação, testes e implantação.

Gabarito: Letra E

4. (FGV / TJ RJ - 2024) Com relação ao modelo incremental do ciclo de vida do software, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() O ciclo de vida incremental é um modelo de desenvolvimento de software que divide o projeto em incrementos entregues ao usuário de forma sequencial.

() O ciclo de vida incremental é um modelo iterativo, pois cada incremento é desenvolvido e entregue ao usuário, e então o processo é repetido para o próximo incremento.

() O ciclo de vida incremental é um modelo mais eficiente do que o modelo em cascata, pois permite que o software seja entregue ao usuário mais rapidamente.

As afirmativas são, respectivamente,

- a) V - V - V.
- b) F - V - V.
- c) F - V - F.
- d) V - F - F.
- e) V - V - F.

Comentários:

(V) No modelo incremental, o sistema é dividido em partes funcionais entregues em sequência, permitindo evolução gradual do software.

(V) Há repetição do processo a cada incremento, com desenvolvimento e entrega sucessivos, o que caracteriza a abordagem iterativa.

(F) Embora permita entregas mais rápidas ao usuário, isso não autoriza afirmar de forma absoluta que seja mais eficiente que o cascata em qualquer contexto.

Gabarito: Letra E

5. (FGV / SES MT - 2024) A engenharia de software tem empregado prototipação para gerar modelos representativos simulados para validação dos requisitos dos usuários. Com relação à prototipação, analise os itens a seguir.

I. É o processo de criação de um protótipo, que dependendo das necessidades do projeto, podem empregar, entre outros: protótipos de papel, wireframes, mockups ou protótipos interativos.

II. No processo de engenharia de requisitos, as fases de eliciação e validação dos requisitos do sistema podem ser auxiliadas por um protótipo.

III. Um protótipo desenvolvido no processo de projeto do sistema pode auxiliar na exploração de soluções de software e no desenvolvimento de uma interface com o usuário para o sistema.

Está correto o que se afirma em

- a) I, II e III.
- b) I e II, apenas.
- c) I e III, apenas.
- d) II e III, apenas.

Comentários:

(I) Correto. Prototipação é justamente a criação de modelos representativos, como papel, wireframes, mockups e protótipos interativos, conforme a necessidade do projeto.

(II) Correto. O protótipo ajuda a levantar, esclarecer e validar requisitos, facilitando o entendimento entre usuários e equipe durante a engenharia de requisitos.

(III) Correto. No projeto do sistema, o protótipo apoia a exploração de soluções e contribui para definir e evoluir a interface com o usuário.

Gabarito: Letra A

6. (FGV / MPE SC - 2022) Um Time de Desenvolvimento de Sistemas de Software (TDSS) decidiu adotar um modelo de processo de software prescritivo como padrão para novos desenvolvimentos de sistemas. O modelo de processo adotado visa à redução do impacto de mudanças em requisitos, no qual a qualidade da documentação não é uma prioridade. O TDSS adotou o modelo de processo:

- a) espiral;
- b) cascata;
- c) unificado;
- d) evolutivo;
- e) prototipagem.

Comentários:

(a) Errado. O modelo espiral é orientado a riscos e iterações, mas não tem como foco principal a baixa prioridade da documentação.

(b) Errado. O cascata é mais rígido, com forte ênfase em fases bem definidas e documentação, o que não combina com o cenário.

(c) Errado. O unificado é iterativo, porém costuma manter preocupação relevante com artefatos e documentação do processo.

(d) Errado. O evolutivo reduz impactos de mudanças, mas a alternativa esperada destaca mais diretamente o uso de protótipos.

(e) Correto. A prototipagem ajuda a acomodar mudanças de requisitos e prioriza a interação e validação rápida, sem foco central na documentação.

Gabarito: Letra E

LISTA DE QUESTÕES

1. (FGV / Câmara Municipal de Fortaleza - 2024) O modelo de desenvolvimento de software que possui ênfase na documentação extensiva e planejamento detalhado que é caracterizado por uma abordagem sequencial e linear, no qual cada fase deve ser concluída antes que a próxima seja iniciada, é conhecido como

- a) Modelo Espiral.
- b) Desenvolvimento Orientado a Testes (TDD).
- c) Modelo Lean.
- d) Modelo em Cascata.
- e) Desenvolvimento Ágil.

2. (FGV / Prefeitura de Cuiabá - 2024) Entre as metodologias prescritivas de desenvolvimento de software, a metodologia em Cascata subdivide o desenvolvimento de um novo software em cinco atividades: Engenharia de Sistemas, Projeto, Codificação, Teste e Manutenção. Analise as afirmativas a seguir relativas à metodologia Cascata.

I - A interação com o cliente ocorre somente nas fases de Projeto e Manutenção.

II - As atividades de Codificação e Testes são executadas em paralelo para reduzir o tempo de desenvolvimento do software.

III - Os requisitos do sistema são determinados na atividade de Engenharia de sistemas.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) III, apenas.
- d) I e II, apenas.
- e) II e III, apenas.

3. (FGV / TJ SE - 2023) O gerente de TI de sua empresa solicitou a implementação de um pequeno sistema para geração da folha salarial. Como será um sistema simples, com regras imutáveis e bem definidas, o analista já definiu os requisitos e efetuou a modelagem do sistema. Após o final da implementação, será iniciada a fase de testes e, por fim, a implantação no setor financeiro. De acordo com esse contexto, o modelo de ciclo de vida adotado foi:

- a) RUP;

- b) modelo incremental;
- c) prototipagem;
- d) modelo evolutivo;
- e) modelo em cascata.

4. (FGV / TJ RJ - 2024) Com relação ao modelo incremental do ciclo de vida do software, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() O ciclo de vida incremental é um modelo de desenvolvimento de software que divide o projeto em incrementos entregues ao usuário de forma sequencial.

() O ciclo de vida incremental é um modelo iterativo, pois cada incremento é desenvolvido e entregue ao usuário, e então o processo é repetido para o próximo incremento.

() O ciclo de vida incremental é um modelo mais eficiente do que o modelo em cascata, pois permite que o software seja entregue ao usuário mais rapidamente.

As afirmativas são, respectivamente,

- a) V - V - V.
- b) F - V - V.
- c) F - V - F.
- d) V - F - F.
- e) V - V - F.

5. (FGV / SES MT - 2024) A engenharia de software tem empregado prototipação para gerar modelos representativos simulados para validação dos requisitos dos usuários. Com relação à prototipação, analise os itens a seguir.

I. É o processo de criação de um protótipo, que dependendo das necessidades do projeto, podem empregar, entre outros: protótipos de papel, wireframes, mockups ou protótipos interativos.

II. No processo de engenharia de requisitos, as fases de elicitação e validação dos requisitos do sistema podem ser auxiliadas por um protótipo.

III. Um protótipo desenvolvido no processo de projeto do sistema pode auxiliar na exploração de soluções de software e no desenvolvimento de uma interface com o usuário para o sistema.

Está correto o que se afirma em

- a) I, II e III.

- b) I e II, apenas.
- c) I e III, apenas.
- d) II e III, apenas.

6. (FGV / MPE SC - 2022) Um Time de Desenvolvimento de Sistemas de Software (TDSS) decidiu adotar um modelo de processo de software prescritivo como padrão para novos desenvolvimentos de sistemas. O modelo de processo adotado visa à redução do impacto de mudanças em requisitos, no qual a qualidade da documentação não é uma prioridade. O TDSS adotou o modelo de processo:

- a) espiral;
- b) cascata;
- c) unificado;
- d) evolutivo;
- e) prototipagem.

GABARITO

- | | | | | | |
|----|---------|----|---------|----|---------|
| 1. | LETRA D | 3. | LETRA E | 5. | LETRA A |
| 2. | LETRA C | 4. | LETRA E | 6. | LETRA E |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.